



Al-Imam University

Computer Science Department

College of Computer and Information Sciences

Design and Implementation of a Compiler Front-End for the HealthScript Language

Students :

Name	ID
Nuha Aedh Alrubayyi	444005638
Dan Riyad Alhumrani	444006468

CONTENTS

Abstract	5
1. Introduction	6
2. Project Objectives	6
3. Project Importance	7
4. General Language Specification	7
Chapter 1: Lexical Analysis Phase	9
1.1 Introduction	9
1.2 Phase 1: Overview	9
1.3 Scanner Objective	11
1.4 Token Definitions	13
1.5 Regular Expressions Used	15
1.6 DFA Diagrams for Token Recognition	17
1.7 Scanner Algorithm	20
1.8 Symbol Table	21
1.9 Lexical Error Handling	22
1.10 Test Cases	24
1.11 Conclusion	29
Chapter 2: Syntax Analysis Phase	30
2.0 Introduction	30
2.1 Parser Objective	30

2.2 Original CFG Grammar	30
2.3 LL(1)-Transformed Grammar	33
Expression Grammar with Precedence	34
2.4 FIRST Sets	36
2.5 FOLLOW Sets.....	37
2.6 LL(1) Parsing Table	40
2.7 Parser Algorithm	42
2.8 Recursive Descent Parsing Procedure.....	43
2.9 Parser Design Note.....	44
2.10 Parse Tree Representation	45
Example Parse Tree.....	45
2.11 Syntax Error Handling	46
2.12 Parser Test Cases.....	48
Syntax Error: Expected ';' but found 'x'	51
2.13 Conclusion.....	56
Chapter 3: Semantic Analysis and Code Generation	57
3.0 Introduction	57
3.1 Phase 3 Objective	57
3.2 Enhanced Symbol Table.....	58
3.3 Semantic Rules	58
3.4 Semantic Error Handling.....	59
3.5 Code Generation Approach.....	59
3.6 Test Cases.....	59

3.7 End-to-End Compilation	64
3.8 Conclusion.....	65
Chapter 4: Overall Conclusion.....	67
4.0 Introduction	67
4.1 Project Summary	67
4.2 Achievements of the Project	67
4.3 Challenges Faced.....	68
4.4 Future Improvements	68
4.5 Final Conclusion	69
Appendix.....	70
Appendix A: Scanner Source Code	70
Appendix B: Parser Source Code	74
Appendix C: Semantic Analyzer and TAC Generator Source Code	81

Abstract

This project presents the design and implementation of a compiler front-end for a custom programming language called HealthScript. The compiler was developed in three phases: lexical analysis, syntax analysis, and semantic analysis with intermediate code generation. Python was used to implement the scanner, parser, semantic analyzer, and Three-Address Code (TAC) generator. The final system is capable of processing HealthScript programs, detecting errors, and generating structured intermediate code through a complete compiler front-end pipeline.

1. Introduction

Compiler construction is one of the most important fields in computer science and software engineering. A compiler translates source code written in a programming language into another form that can be processed and executed by a computer system. The compilation process is usually divided into several phases such as lexical analysis, syntax analysis, semantic analysis, and code generation.

This project focuses on designing and implementing a compiler front-end for a custom domain-specific language called HealthScript. The language was designed to simplify programming tasks in healthcare environments by providing readable syntax and easy-to-understand program structures suitable for non-technical users.

The compiler front-end was implemented entirely using Python without relying on external compiler-generation tools. The project includes three main phases. The first phase implements lexical analysis using a scanner and regular expressions. The second phase focuses on syntax analysis using LL(1) grammar transformation and recursive descent parsing. The third phase introduces semantic analysis and intermediate code generation using Three-Address Code (TAC).

The final implementation demonstrates a complete compiler front-end pipeline capable of processing HealthScript programs while detecting lexical, syntax, and semantic errors with meaningful error messages.

2. Project Objectives

The main objectives of this project are:

1. To design a custom programming language called HealthScript.
2. To implement a lexical analyzer for token recognition and lexical error detection.

3. To design a Context-Free Grammar (CFG) and transform it into LL(1) form.
4. To implement a recursive descent parser for syntax analysis.
5. To construct FIRST and FOLLOW sets for predictive parsing.
6. To implement semantic analysis for declaration checking and type validation.
7. To generate intermediate code using Three-Address Code (TAC).
8. To integrate all compiler phases into a complete compiler front-end pipeline.

3. Project Importance

This project demonstrates the practical application of compiler construction concepts through the implementation of a complete compiler front-end system. It combines important compiler design techniques such as lexical analysis, parsing, semantic checking, symbol table management, and intermediate code generation.

The HealthScript language also demonstrates the usefulness of domain-specific programming languages in simplifying technical tasks for specialized fields such as healthcare. The language focuses on readability and simplified syntax to make programming more accessible for non-technical users.

In addition, the project provides practical experience in implementing scanners, parsers, semantic analyzers, and code generators using Python. These components represent the core foundation of modern programming language processors and compiler systems.

4. General Language Specification

HealthScript is a domain-specific scripting language designed to simplify programming tasks in healthcare environments. The language focuses on readability, simplicity, and structured syntax to support healthcare professionals and non-technical users.

The language supports basic data types including int, float, string, and bool. Variables must be declared using the var keyword before use. HealthScript also supports arithmetic, relational, and logical operators for calculations and decision-making operations.

Several control structures are included in the language, such as conditional statements and loop statements. Programs begin with the keyword start and terminate using the keyword finish. Functions, Boolean expressions, and input/output operations are also supported.

The language is case-insensitive and uses simplified syntax rules to improve readability and reduce complexity. The compiler front-end processes HealthScript programs through lexical analysis, syntax analysis, semantic analysis, and intermediate code generation phases.

Chapter 1: Lexical Analysis Phase

1.1 Introduction

Lexical analysis represents the first stage of the HealthScript compiler front-end. In this phase, the source code is scanned and converted into a sequence of tokens that can be processed by later compiler phases such as syntax analysis and semantic analysis. The lexical analyzer is responsible for identifying the smallest meaningful units of the language, including keywords, identifiers, literals, operators, and delimiters.

The scanner developed in this project was implemented using Python and regular expressions without relying on external lexer-generation tools. The implementation processes the input program character by character while maintaining accurate line numbering and structured token generation.

In addition to token recognition, the lexical analyzer also performs symbol table construction and lexical error detection. Different types of lexical errors such as invalid symbols, malformed numeric values, identifier length violations, and unterminated strings or comments are detected and reported with descriptive messages and corresponding line numbers.

This chapter presents the complete implementation of the lexical analysis phase, including token definitions, regular expressions, DFA diagrams, scanner algorithms, symbol table generation, error handling mechanisms, and test case validation.

1.2 Phase 1: Overview

The first phase of the compiler project focuses on Lexical Analysis, which involves building a scanner that reads the source code written in HealthScript and converts it

into a sequence of tokens. This phase is essential because it forms the foundation for subsequent stages such as syntax and semantic analysis.

The scanner developed in this project was implemented from scratch using Python, without relying on any external lexer-generation tools. Its primary function is to process the input program character by character and group sequences of characters into meaningful tokens such as keywords, identifiers, operators, literals, and delimiters.

To achieve accurate token recognition, the scanner utilizes regular expressions (RE) to define patterns for each token type. For example, identifiers follow the pattern of starting with a letter and optionally including digits, while numeric literals are categorized into integers and floating-point numbers with specific constraints. String literals are recognized as sequences of characters enclosed within double quotation marks.

The scanner is designed to be case-insensitive, meaning that all identifiers and keywords are processed in a uniform manner regardless of their letter casing. This is achieved by converting identifier values to lowercase before checking them against the predefined list of reserved keywords.

One of the key responsibilities of the lexical analyzer is to generate detailed output that includes:

- The total number of lexemes identified in the source code.
- A complete list of tokens, where each token includes its type, line number, and value.
- A symbol table that stores all identifiers along with the line number of their first occurrence.

- A list of lexical errors detected during scanning.

Error handling is an important component of this phase. The scanner is capable of detecting and reporting various types of lexical errors, including:

- Invalid symbols that do not belong to the language.
- Identifiers that exceed the maximum allowed length.
- Malformed numeric literals that violate the defined constraints.
- Unterminated string literals.
- Unterminated block comments.

Each error is reported with a clear and descriptive message along with the corresponding line number, making it easier to debug the source code.

Additionally, the scanner properly handles whitespace and comments by ignoring them during token generation, while still maintaining accurate line numbering. This ensures that the output is both clean and informative.

To validate the correctness of the scanner, multiple test cases were used, including both valid programs and programs containing deliberate lexical errors. These test cases demonstrate the scanner's ability to correctly identify tokens and detect errors as required.

In conclusion, the lexical analysis phase successfully transforms HealthScript source code into a structured sequence of tokens while providing detailed error reporting and symbol table management. This serves as a solid foundation for the next phase of the compiler, which is syntax analysis.

1.3 Scanner Objective

The main objective of the scanner (lexical analyzer) in this project is to read a HealthScript source program and convert it into a structured stream of tokens that can

be used by the next phase of the compiler (syntax analysis). The scanner acts as the first stage of the compilation process, where raw input text is analyzed and broken down into meaningful components.

The scanner is designed to process the input program sequentially, character by character, and group sequences of characters into valid lexemes such as keywords, identifiers, literals, operators, and delimiters. Each lexeme is then classified into a corresponding token type. This transformation simplifies the input for the parser by removing unnecessary details such as whitespace and comments while preserving the logical structure of the program.

Another important objective of the scanner is to maintain accurate tracking of line numbers throughout the scanning process. This allows the system to provide precise and meaningful error messages whenever a lexical error is detected. Line number tracking is especially important for debugging and improving code readability.

In addition to token generation, the scanner is responsible for building and maintaining a symbol table. The symbol table stores all identifiers encountered in the program along with the line number of their first occurrence. This structure is essential for later phases of the compiler, such as semantic analysis.

Error detection is also a key objective of the scanner. The implemented scanner is capable of identifying several types of lexical errors, including invalid symbols, identifiers that exceed the maximum allowed length, malformed numeric literals, and unterminated string literals or block comments. Each error is reported clearly with its corresponding line number to assist users in correcting their code.

Overall, the scanner ensures that the input program is transformed into a clean, organized, and error-checked sequence of tokens, forming a reliable foundation for further analysis in the compiler pipeline.

1.4 Token Definitions

In HealthScript, tokens represent the smallest meaningful units of the source code. Each token is classified based on its role in the language and is identified using regular expressions. The scanner categorizes tokens into several types, each corresponding to a specific syntactic element.

The first category is Keywords, which are reserved words that have predefined meanings in the language and cannot be used as identifiers. Examples include start, finish, if, then, else, while, repeat, var, int, float, string, bool, read, print, func, return, and, or, and not. These keywords define the structure and behavior of HealthScript programs.

The second category is Identifiers, which represent variable names, function names, and other user-defined entities. Identifiers must begin with an English letter and can be followed by letters or digits. Additionally, the maximum allowed length for an identifier is eight characters, ensuring simplicity and consistency within the language.

The third category includes Numeric Literals, which are divided into integers and floating-point numbers. Integer literals consist of digits with an optional sign, while floating-point literals include a decimal point and digits on both sides. HealthScript enforces a constraint that numeric values must not exceed eight digits before the decimal point, and any violation of this rule is treated as a lexical error.

Another important category is String Literals, which are sequences of characters enclosed within double quotation marks. These are typically used for output messages

and textual data. The scanner ensures that all string literals are properly terminated; otherwise, an error is reported.

The language also includes Operators, which are used to perform operations on values. These are divided into arithmetic operators (+, -, *, /, %, ^), relational operators (==, !=, <, >, <=, >=), and logical operators (and, or, not). The assignment operator (=) is also included for assigning values to variables.

Additionally, Delimiters are used to separate different elements of the program. These include symbols such as parentheses, commas, semicolons, and braces. Delimiters help define the structure and organization of statements and expressions.

Finally, the scanner also recognizes Comments and Whitespace, which are ignored during token generation. Comments can be either single-line (starting with //) or block comments enclosed between /* and */. Although these elements are not converted into tokens, they are handled carefully to maintain accurate line numbering.

Each of these token types is defined using regular expressions, enabling the scanner to efficiently and accurately classify different parts of the source code. This structured classification is essential for ensuring that the program is correctly interpreted in later stages of compilation.

Table 1 : Summary of Token Types

Token Type	Description	Example
KEYWORD	Reserved words with predefined meaning	start, if, var
IDENTIFIER	User-defined names for variables/functions	ptTemp, heartRt

INTEGER	Whole numbers (max 8 digits)	10, -5
FLOAT	Decimal numbers (max 8 digits before decimal)	36.5, -12.75
STRING	Text enclosed in double quotes	"Hello"
REL_OP	Relational operators	==, !=, >, <=
ARITH_OP	Arithmetic and assignment operators	+, -, *, /, =
DELIMITER	Symbols for structure and separation	(,), ;, ,
LINE_COMMENT	Single-line comments (ignored)	// comment
BLOCK_COMMENT	Multi-line comments (ignored)	/* comment */
WHITESPACE	Spaces and tabs (ignored)	(space), tab
NEWLINE	Line breaks (used for tracking lines)	\n
UNKNOWN	Invalid or unrecognized symbols	@, #
ERROR	Invalid tokens or malformed inputs	long identifier, bad number

1.5 Regular Expressions Used

Regular Expressions (RE) are used in the scanner to define patterns for recognizing different token types in HealthScript. These patterns allow the lexical analyzer to efficiently match sequences of characters in the source code and classify them into appropriate tokens.

Each token type is associated with a specific regular expression. The scanner processes the input by matching these expressions in a predefined order, ensuring that more specific patterns (such as relational operators or malformed numbers) are

detected before more general ones. This ordering is critical to avoid incorrect classification.

Identifiers are defined using a pattern that ensures they start with a letter followed by letters or digits. Numeric literals are divided into integers and floating-point numbers, with constraints applied to limit the number of digits. Special patterns are also used to detect malformed numbers that exceed the allowed limits. String literals are recognized as sequences enclosed within double quotation marks, while additional patterns are included to detect unterminated strings.

Operators and delimiters are identified using character-based expressions, while comments are handled using patterns that match single-line and block comment formats. The scanner also includes a fallback pattern to detect any unknown or invalid symbols.

The following table summarizes the main regular expressions used in the scanner:

Table 2 : regular expressions

Token Type	Regular Expression	Description
BLOCK_COMMENT	<code>\/*(?:.</code>	<code>\n)*?*/</code>
UNTERMINATED_BLOCK	<code>\/*(?:.</code>	<code>\n)*</code>
LINE_COMMENT	<code>//(?:\n)*</code>	Matches single-line comments
STRING	<code>"(?:\n)*"</code>	Matches valid string literals
UNTERMINATED_STRING	<code>"(?:\n)*</code>	Detects missing closing quotation
MALFORMED_FLOAT	<code>[+-]?d{9,}\.d+</code>	Detects floats exceeding digit

		limit
MALFORMED_INTEGER	[+-]?d{9,}	Detects integers exceeding digit limit
FLOAT	[+-]?d{1,8}\.d+	Matches valid floating-point numbers
INTEGER	[+-]?d{1,8}	Matches valid integers
REL_OP	== != <= >= < >	Matches relational operators
ARITH_OP	[+\-*\\/\\^=]	Matches arithmetic and assignment operators
DELIMITER	[.\\(\\),\\{\\}\\;:]	Matches delimiters
IDENTIFIER	[a-zA-Z][a-zA-Z0-9]*	Matches identifiers
NEWLINE	\\n	Tracks line breaks
WHITESPACE	[\\t]+	Matches spaces and tabs
UNKNOWN	.	Matches any invalid character

These regular expressions form the backbone of the scanner, enabling it to accurately recognize tokens and detect lexical errors.

1.6 DFA Diagrams for Token Recognition

This section presents deterministic finite automata (DFA) used to recognize key token types in HealthScript.

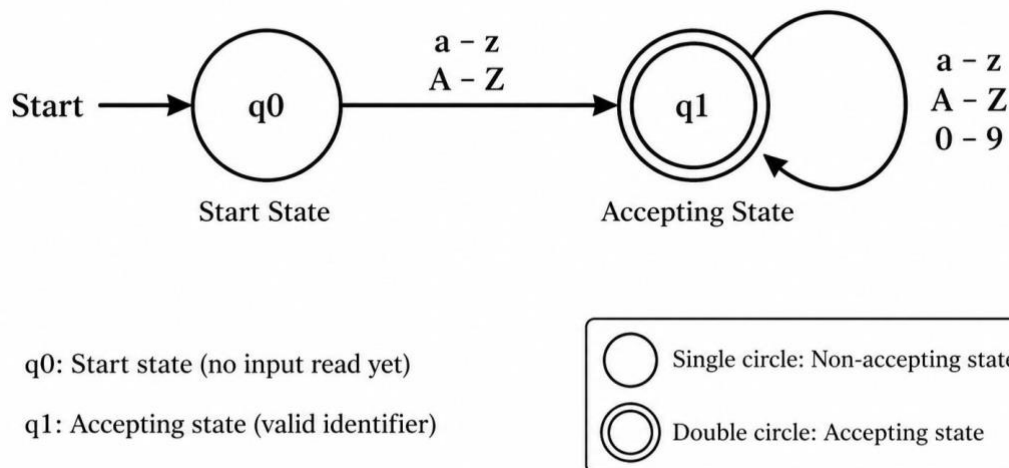


Figure 1: DFA for Identifier

This DFA recognizes identifiers that begin with a letter followed by letters or digits.

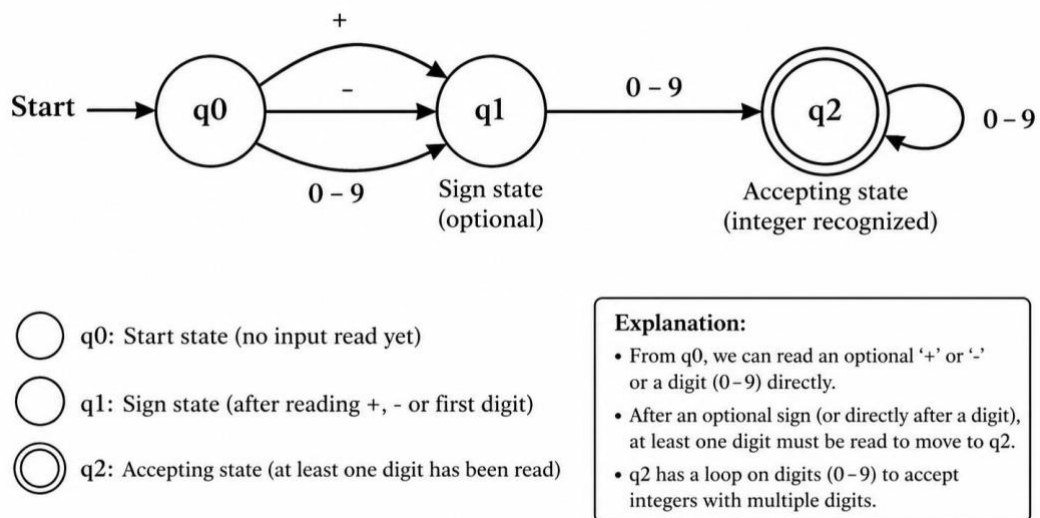
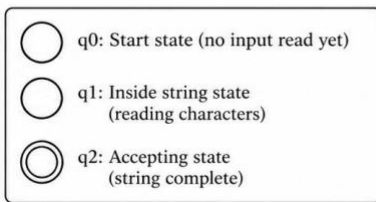
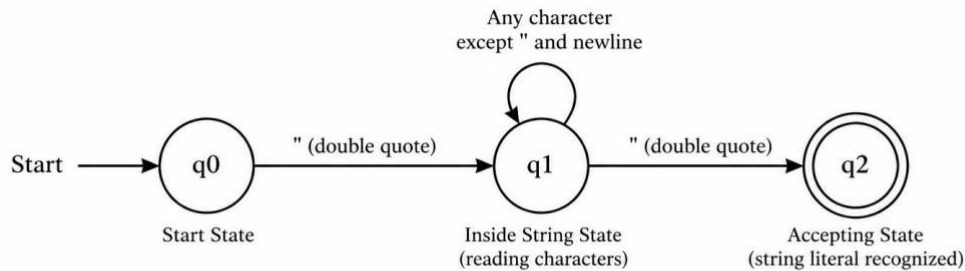


Figure 2: DFA for Integer

This DFA recognizes integer literals with an optional positive or negative sign followed by one or more digits.

DFA for String Literals



Explanation:

- The DFA starts in q0.
- On reading the opening double quote ("), it moves to q1.
- In q1, it accepts any character except a double quote (") or newline (line break). It stays in q1 for each such character.
- On reading the closing double quote ("), it moves to q2, which is the accepting state.
- If a newline or end of input is reached in q1 without a closing double quote, the string is considered invalid.

Figure 3: DFA for String Literal

This DFA is used to recognize string literals in HealthScript.

The process begins at the start state (q0). When the scanner reads an opening double quote ("), it transitions to state (q1), which represents being inside a string. In state (q1), the DFA accepts any character except a newline or another double quote. It remains in this state as long as valid characters are read. When a closing double quote (") is encountered, the DFA moves to the accepting state (q2), indicating that a valid string literal has been recognized. If the input ends or a newline is reached before a closing double quote is found, the string is considered invalid and results in a lexical error.

1.7 Scanner Algorithm

The scanner algorithm is designed to process the input source code sequentially and convert it into a list of tokens while detecting any lexical errors. The implementation follows a structured approach that ensures correctness, efficiency, and clarity.

The algorithm begins by initializing key data structures, including a list to store tokens, a symbol table to track identifiers, a list for errors, and a counter for the total number of lexemes. A predefined set of keywords is also initialized to distinguish reserved words from identifiers.

Next, all regular expressions are combined into a single pattern using named groups. This allows the scanner to match multiple token types in a single pass through the input. The program then iterates over the source code using pattern matching.

During each iteration, the scanner identifies the type of token (token kind) and its corresponding value. If the token is a newline character, the line counter is incremented. If the token is whitespace or a comment, it is ignored. For block comments, the scanner also counts any newlines within the comment to maintain accurate line numbering.

If the matched token corresponds to an error condition—such as an unterminated string, malformed number, or unknown symbol—the scanner records an appropriate error message along with the line number and continues processing the rest of the input.

When a valid token is detected, the scanner checks whether it is an identifier. If so, it converts the value to lowercase and compares it with the list of keywords. If it matches a keyword, it is classified accordingly. Otherwise, it is treated as an

identifier, provided it does not exceed the maximum length. Valid identifiers are added to the symbol table if they appear for the first time.

For all other token types, the scanner directly appends the token (including its type, line number, and value) to the token list. The total number of lexemes is incremented for each valid token identified.

After processing the entire input, the scanner returns the total number of lexemes, the complete token list, the symbol table, and the list of errors.

In summary, the scanner algorithm efficiently transforms raw source code into a structured representation while ensuring accurate error detection and reporting. This structured output is essential for the next phase of the compiler, which performs syntax analysis.

1.8 Symbol Table

The Symbol Table is a fundamental data structure used in the lexical analysis phase to store information about identifiers encountered in the source code. In this project, the symbol table is implemented as a dictionary where each identifier is stored along with the line number of its first occurrence.

During the scanning process, whenever an identifier is recognized and validated (i.e., it meets the naming rules and does not exceed the maximum allowed length), the scanner checks whether it already exists in the symbol table. If the identifier appears for the first time, it is added to the table along with the current line number. If it has already been recorded, no duplication occurs, ensuring that each identifier is stored only once.

The symbol table serves several important purposes. First, it provides a clear overview of all user-defined names used in the program, which helps in understanding

the structure of the code. Second, it plays a critical role in later phases of the compiler, especially semantic analysis, where information such as variable declarations, scope, and type checking will be required.

For example, in a sample HealthScript program, identifiers such as `ptTemp`, `heartRt`, and `errInt` are stored in the symbol table along with the line numbers where they first appear. This allows the compiler to reference these identifiers efficiently during subsequent processing.

The design of the symbol table in this phase is kept simple, focusing only on storing the identifier name and its first occurrence. However, it can be extended in future phases to include additional attributes such as data type, scope level, and memory location.

Overall, the symbol table enhances the organization and management of identifiers, making it an essential component of the lexical analyzer.

1.9 Lexical Error Handling

Error handling is a crucial component of the lexical analysis phase, as it ensures that any invalid or improperly formatted elements in the source code are detected and reported clearly. The HealthScript scanner is designed to identify different types of lexical errors while continuing the scanning process to capture all possible issues in a single run.

One of the main errors detected by the scanner is the presence of invalid symbols. Any character that does not match the defined token patterns is classified as an unknown token and reported as a lexical error. This prevents unsupported or unintended characters from being processed further.

The scanner also checks for identifier length violations. According to the language rules, identifiers must not exceed eight characters. If a longer identifier is encountered, the scanner reports it as an error while still continuing the analysis of the remaining code.

Another important type of error is related to malformed numeric literals. The scanner validates both integers and floating-point numbers to ensure they follow the specified constraints. If a number exceeds the allowed digit limit or does not match the correct format, it is reported as a malformed number.

In addition, the scanner detects unterminated string literals, which occur when a string begins with a quotation mark but is not properly closed. This type of error is clearly reported along with the partial string value to help identify the issue.

The scanner also identifies unterminated block comments, where a comment starts with `/*` but does not end with `*/`. This ensures that incomplete comments do not affect the parsing of the remaining code.

Each detected error is stored in a dedicated error list and reported with a descriptive message and the exact line number where it occurred. This allows users to quickly locate and correct issues in their code.

Importantly, the scanner is designed to continue processing even after encountering errors, rather than stopping execution. This approach improves usability by allowing multiple errors to be identified in a single run.

Overall, the error handling mechanism enhances the reliability of the scanner by ensuring that all lexical issues are detected, clearly reported, and easy to debug.

1.10 Test Cases

1. Test Case 1 : Valid Program

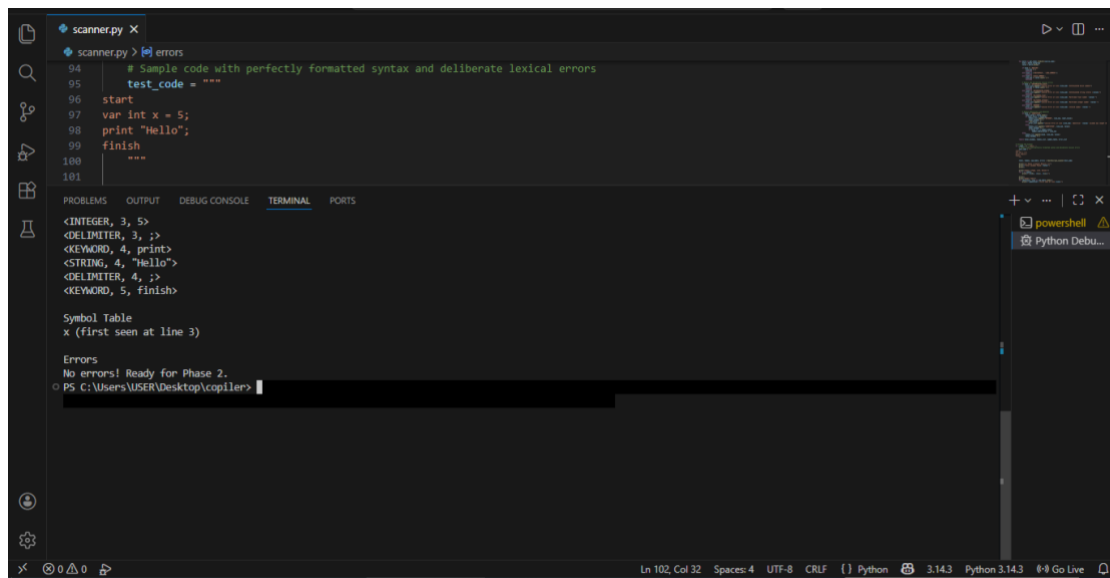
Code

start

var int x = 5;

print "Hello";

finish



Output:

The scanner successfully analyzed the program. All tokens were correctly identified, the symbol table was generated, and no lexical errors were found.

2. Test Case 2 : Valid Program

Code

start

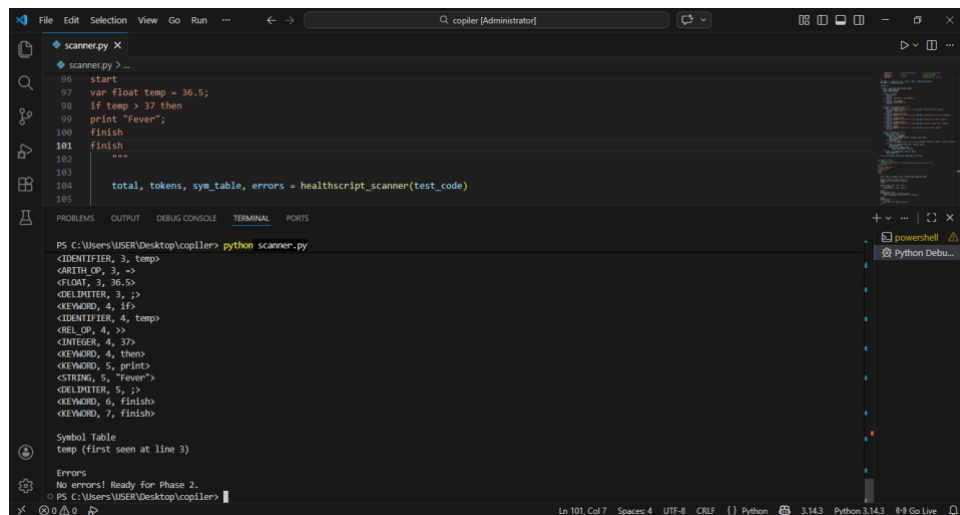
var float temp = 36.5;

if temp > 37 then

print "Fever";

finish

finish



The screenshot shows a VS Code editor with a file named 'scanner.py'. The code in the editor is as follows:

```
96 start
97 var float temp = 36.5;
98 if temp > 37 then
99   print "Fever";
100 finish
101 finish
102
103
104 total, tokens, sym_table, errors = healthscript_scanner(test_code)
105
```

The terminal output shows the results of the scanner's execution:

```
PS C:\Users\USER\Desktop\copiler> python scanner.py
<IDENTIFIER, 3, temp>
<ARITH_OP, 3, =>
<FLOAT, 3, 36.5>
<DELIMITER, 3, ;>
<KEYWORD, 4, if>
<IDENTIFIER, 4, temp>
<REL_OP, 4, >>
<INTEGER, 4, 37>
<KEYWORD, 4, then>
<KEYWORD, 5, print>
<STRING, 5, "Fever">
<DELIMITER, 5, ;>
<KEYWORD, 6, finish>
<KEYWORD, 7, finish>

Symbol Table
temp (first seen at line 3)

Errors
No errors! Ready for Phase 2.
PS C:\Users\USER\Desktop\copiler>
```

Output:

The scanner successfully processed the input program. All tokens were recognized correctly, and no lexical errors were detected.

3. Test Case 3 : Valid Program

Code

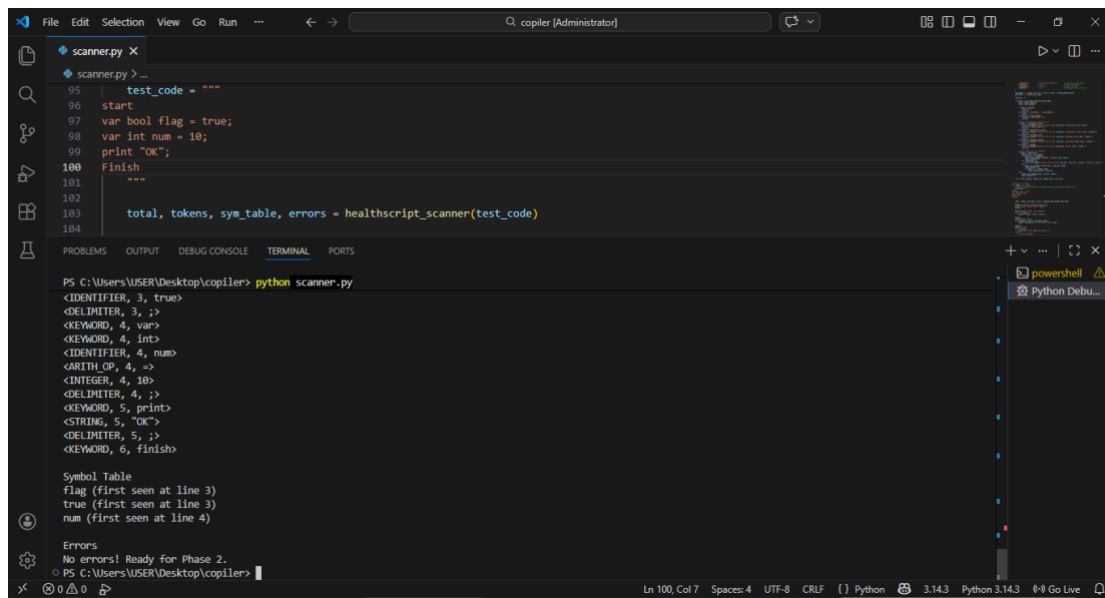
start

var bool flag = true;

var int num = 10;

print "OK";

finish



The screenshot shows a Visual Studio Code editor window with a file named `scanner.py`. The code in the editor is as follows:

```
95 test_code = """
96 start
97 var bool flag = true;
98 var int num = 10;
99 print "OK";
100 Finish
101 """
102
103 total, tokens, sym_table, errors = healthscript_scanner(test_code)
104
```

The terminal at the bottom shows the command `python scanner.py` being executed. The output displays the tokens and symbol table generated by the scanner:

```
PS C:\Users\USER\Desktop\copiler> python scanner.py
<IDENTIFIER, 3, true>
<DELIMITER, 3, ;>
<KEYWORD, 4, var>
<KEYWORD, 4, int>
<IDENTIFIER, 4, num>
<ARITH_OP, 4, =>
<INTEGER, 4, 10>
<DELIMITER, 4, ;>
<KEYWORD, 5, print>
<STRING, 5, "OK">
<DELIMITER, 5, ;>
<KEYWORD, 6, finish>

Symbol Table
flag (first seen at line 3)
true (first seen at line 3)
num (first seen at line 4)

Errors
No errors! Ready for Phase 2.
PS C:\Users\USER\Desktop\copiler>
```

Output:

The scanner correctly generated tokens and symbol table entries. No lexical errors were found.

4. Test Case 4 : Program with Lexical Errors

Code

start

```
var int veryLongVariableName = 10;
```

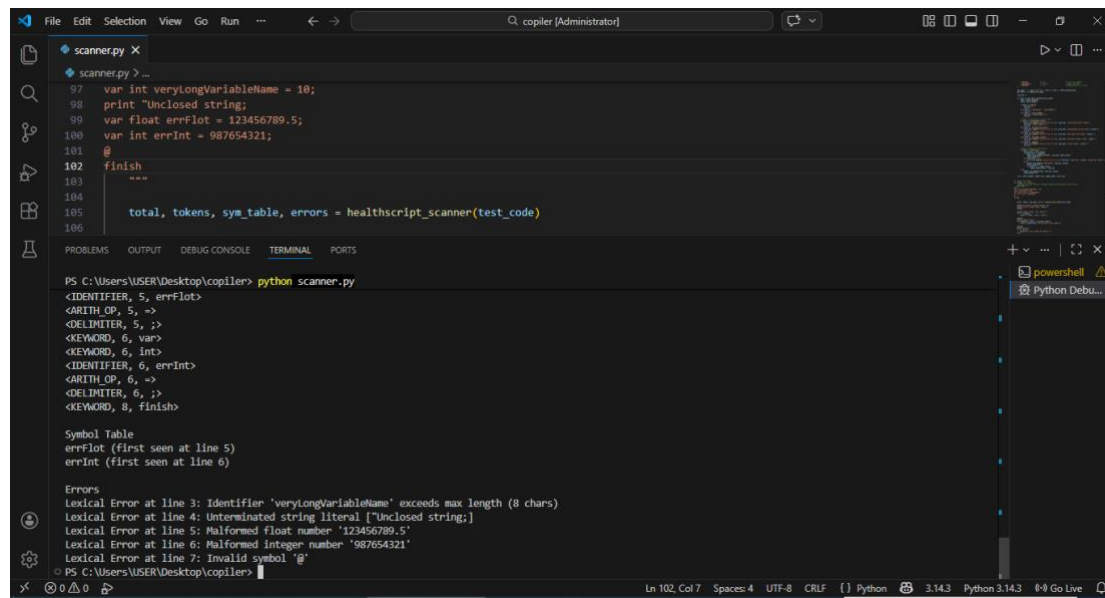
```
print "Unclosed string;
```

```
var float errFlot = 123456789.5;
```

```
var int errInt = 987654321;
```

```
@
```

finish



The screenshot shows a VS Code editor window with a file named 'scanner.py'. The code in the editor is as follows:

```
97 var int veryLongVariableName = 10;
98 print "Unclosed string;
99 var float errFlot = 123456789.5;
100 var int errInt = 987654321;
101 @
102 finish
103 """
104
105 total, tokens, sym_table, errors = healthscript_scanner(test_code)
106
```

The terminal output shows the execution of the scanner and the detected errors:

```
PS C:\Users\USER\Desktop\copiler> python scanner.py
<IDENTIFIER, 5, errFlot>
<ARITH_OP, 5, =>
<DELIMITER, 5, ;>
<KEYWORD, 6, var>
<KEYWORD, 6, int>
<IDENTIFIER, 6, errInt>
<ARITH_OP, 6, =>
<DELIMITER, 6, ;>
<KEYWORD, 8, finish>

Symbol Table
errFlot (first seen at line 5)
errInt (first seen at line 6)

Errors
Lexical Error at line 3: Identifier 'veryLongVariableName' exceeds max length (8 chars)
Lexical Error at line 4: Unterminated string literal ["Unclosed string;]
Lexical Error at line 5: Malformed float number '123456789.5'
Lexical Error at line 6: Malformed integer number '987654321'
Lexical Error at line 7: Invalid symbol '@'
```

Output:

The scanner detected multiple lexical errors, including:

- Identifier exceeding maximum length
- Unterminated string literal
- Malformed float number
- Malformed integer number

- Invalid symbol

5. Test Case 5 : Program with Lexical Errors

Code

start

var int x = 123456789;

print "Hello

finish

```

scanner.py
# Sample code with perfectly formatted syntax and deliberate lexical errors
test_code = """
start
var int x = 123456789;
print "Hello
finish
"""

total, tokens, sym_table, errors = healthscript_scanner(test_code)

```

```

PS C:\Users\USER\Desktop\copiler> python scanner.py
PS C:\Users\USER\Desktop\copiler> python scanner.py
=== Phase 1 Scanner Results ===
Total Lexemes Found: 8

Tokens (<Type, Line, Value>)
<KEYWORD, 2, start>
<KEYWORD, 3, var>
<KEYWORD, 3, int>
<IDENTIFIER, 3, x>
<ARITH_OP, 3, =>
<DELIMITER, 3, ;>
<KEYWORD, 4, print>
<KEYWORD, 5, finish>

Symbol Table
x (first seen at line 3)

Errors
Lexical Error at line 3: Malformed integer number '123456789'
Lexical Error at line 4: Unterminated string literal ["Hello]
PS C:\Users\USER\Desktop\copiler>

```

Output:

The scanner reported lexical errors, including:

- Malformed integer number
- Unterminated string literal

1.11 Conclusion

In this phase of the project, a lexical analyzer (scanner) for the HealthScript language was successfully designed and implemented. The scanner is capable of reading source code and transforming it into a structured stream of tokens, which serves as the foundation for the subsequent phases of the compiler.

The implementation supports all required token types, including keywords, identifiers, numeric literals, string literals, operators, and delimiters. In addition, the scanner effectively ignores comments and whitespace while maintaining accurate line numbering, ensuring clean and meaningful output.

A symbol table was also constructed to store identifiers along with the line numbers of their first occurrence. This structure will be essential in later stages of the compiler, particularly in semantic analysis.

Furthermore, the scanner demonstrates strong error detection capabilities. It successfully identifies invalid symbols, identifiers exceeding the allowed length, malformed numeric values, and unterminated strings or comments. All errors are reported with clear messages and corresponding line numbers, making debugging easier.

The correctness and reliability of the scanner were verified through multiple test cases, including both valid programs and programs containing deliberate lexical errors. The results confirm that the scanner meets all the requirements specified for Phase 1.

Overall, the lexical analysis phase provides a solid and reliable foundation for the next stage of the compiler project, which involves syntax analysis and parser implementation.

Chapter 2: Syntax Analysis Phase

2.0 Introduction

Syntax analysis is the second phase of the HealthScript compiler front-end. This phase receives the token stream generated by the lexical analyzer and verifies whether the program follows the grammar rules of the language. The parser checks the structure of declarations, assignments, expressions, input/output statements, control structures, function declarations, and function calls.

In this project, syntax analysis was implemented using a recursive descent parser based on an LL(1)-transformed grammar. The original Context-Free Grammar (CFG) was transformed to remove left recursion and make it suitable for predictive parsing. FIRST and FOLLOW sets were also constructed to support parsing decisions.

This chapter presents the parser objective, original CFG grammar, LL(1)-transformed grammar, FIRST and FOLLOW sets, LL(1) parsing table, recursive descent parsing procedure, parse tree representation, syntax error handling, and parser test cases.

2.1 Parser Objective

The objective of Phase 2 is to design and implement a syntax analyzer for the HealthScript language. The parser receives the token stream generated by the lexical analyzer in Phase 1 and checks whether the program follows the grammar rules of the language. In this phase, a Context-Free Grammar is designed, transformed into an LL(1) suitable form, and used to build a predictive parser. The parser should accept valid HealthScript programs and report meaningful syntax errors for invalid programs.

2.2 Original CFG Grammar

```

<Program> → start <StmtList> finish

<StmtList> → <Stmt> <StmtList> | ε

<Stmt> → <VarDecl>
        | <AssignStmt>
        | <PrintStmt>
        | <ReadStmt>
        | <IfStmt>
        | <WhileStmt>
        | <FuncDecl>
        | <FuncCall>

<VarDecl> → var <Type> id ;

<Type> → int | float | string | bool

<AssignStmt> → id = <Expr> ;

<PrintStmt> → print <Expr> ;

<ReadStmt> → read id ;

<IfStmt> → if <BoolExpr> then <StmtList> <ElsePart>
finish

<ElsePart> → else <StmtList> | ε

<WhileStmt> → while <BoolExpr> do <StmtList> finish

<FuncDecl> → func <ReturnType> id ( <ParamList> )
<StmtList> finish

<ReturnType> → int | float | string | bool | void

<ParamList> → <Param> <ParamRest> | ε

<ParamRest> → , <Param> <ParamRest> | ε

<Param> → <Type> id

```

```

<FuncCall> → do id ( <ArgList> ) ;

<ArgList> → <Expr> <ArgRest> | ε

<ArgRest> → , <Expr> <ArgRest> | ε

<BoolExpr> → <Expr> <RelOp> <Expr>

           | <BoolExpr> or <BoolTerm>

           | <BoolTerm>

<BoolTerm> → <BoolTerm> and <BoolFactor>

           | <BoolFactor>

<BoolFactor> → not <BoolFactor>

              | <Expr> <RelOp> <Expr>

              | true

              | false

<RelOp> → == | != | < | > | <= | >=

<Expr> → <Expr> + <Term>

        | <Expr> - <Term>

        | <Term>

<Term> → <Term> * <Power>

        | <Term> / <Power>

        | <Term> % <Power>

        | <Power>

<Power> → <Factor> ^ <Power>

        | <Factor>

<Factor> → id

          | integer

```



```

| float
| string
| true
| false
| ( <Expr> )
| + <Factor>
| - <Factor>

```

The original grammar describes the general structure of HealthScript programs. However, some expression rules contain left recursion, especially in arithmetic and Boolean expressions. Therefore, the grammar must be transformed to remove left recursion and become suitable for LL(1) predictive parsing.

2.3 LL(1)-Transformed Grammar

```
<Program> → start <StmtList> finish
```

```
<StmtList> → <Stmt> <StmtList> | ε
```

```
<Stmt> → <VarDecl>
```

```
    | <AssignStmt>
```

```
    | <PrintStmt>
```

```
    | <ReadStmt>
```

```
    | <IfStmt>
```

```
    | <WhileStmt>
```

```
    | <FuncDecl>
```

```
    | <FuncCall>
```

```
<VarDecl> → var <Type> id ;
```

```
<Type> → int | float | string | bool
```

$\langle \text{AssignStmt} \rangle \rightarrow \text{id} = \langle \text{Expr} \rangle ;$
 $\langle \text{PrintStmt} \rangle \rightarrow \text{print} \langle \text{Expr} \rangle ;$
 $\langle \text{ReadStmt} \rangle \rightarrow \text{read id} ;$
 $\langle \text{IfStmt} \rangle \rightarrow \text{if} \langle \text{BoolExpr} \rangle \text{ then } \langle \text{StmtList} \rangle \langle \text{ElsePart} \rangle \text{ finish}$
 $\langle \text{ElsePart} \rangle \rightarrow \text{else } \langle \text{StmtList} \rangle \mid \varepsilon$
 $\langle \text{WhileStmt} \rangle \rightarrow \text{while } \langle \text{BoolExpr} \rangle \text{ do } \langle \text{StmtList} \rangle \text{ finish}$
 $\langle \text{FuncDecl} \rangle \rightarrow \text{func } \langle \text{ReturnType} \rangle \text{ id } (\langle \text{ParamList} \rangle) \langle \text{StmtList} \rangle \text{ finish}$
 $\langle \text{ReturnType} \rangle \rightarrow \text{int} \mid \text{float} \mid \text{string} \mid \text{bool} \mid \text{void}$
 $\langle \text{ParamList} \rangle \rightarrow \langle \text{Param} \rangle \langle \text{ParamRest} \rangle \mid \varepsilon$
 $\langle \text{ParamRest} \rangle \rightarrow , \langle \text{Param} \rangle \langle \text{ParamRest} \rangle \mid \varepsilon$
 $\langle \text{Param} \rangle \rightarrow \langle \text{Type} \rangle \text{id}$
 $\langle \text{FuncCall} \rangle \rightarrow \text{do id } (\langle \text{ArgList} \rangle) ;$
 $\langle \text{ArgList} \rangle \rightarrow \langle \text{Expr} \rangle \langle \text{ArgRest} \rangle \mid \varepsilon$
 $\langle \text{ArgRest} \rangle \rightarrow , \langle \text{Expr} \rangle \langle \text{ArgRest} \rangle \mid \varepsilon$

Expression Grammar with Precedence

$\langle \text{BoolExpr} \rangle \rightarrow \langle \text{BoolTerm} \rangle \langle \text{BoolExpr}' \rangle$
 $\langle \text{BoolExpr}' \rangle \rightarrow \text{or } \langle \text{BoolTerm} \rangle \langle \text{BoolExpr}' \rangle \mid \varepsilon$
 $\langle \text{BoolTerm} \rangle \rightarrow \langle \text{BoolFactor} \rangle \langle \text{BoolTerm}' \rangle$
 $\langle \text{BoolTerm}' \rangle \rightarrow \text{and } \langle \text{BoolFactor} \rangle \langle \text{BoolTerm}' \rangle \mid \varepsilon$
 $\langle \text{BoolFactor} \rangle \rightarrow \text{not } \langle \text{BoolFactor} \rangle$
 $\quad \mid \langle \text{Expr} \rangle \langle \text{RelPart} \rangle$
 $\quad \mid \text{true}$
 $\quad \mid \text{false}$

$\langle \text{RelPart} \rangle \rightarrow \langle \text{RelOp} \rangle \langle \text{Expr} \rangle \mid \varepsilon$
 $\langle \text{RelOp} \rangle \rightarrow == \mid != \mid < \mid > \mid <= \mid >=$
 $\langle \text{Expr} \rangle \rightarrow \langle \text{Term} \rangle \langle \text{Expr}' \rangle$
 $\langle \text{Expr}' \rangle \rightarrow + \langle \text{Term} \rangle \langle \text{Expr}' \rangle$
 $\quad \mid - \langle \text{Term} \rangle \langle \text{Expr}' \rangle$
 $\quad \mid \varepsilon$
 $\langle \text{Term} \rangle \rightarrow \langle \text{Power} \rangle \langle \text{Term}' \rangle$
 $\langle \text{Term}' \rangle \rightarrow * \langle \text{Power} \rangle \langle \text{Term}' \rangle$
 $\quad \mid / \langle \text{Power} \rangle \langle \text{Term}' \rangle$
 $\quad \mid \% \langle \text{Power} \rangle \langle \text{Term}' \rangle$
 $\quad \mid \varepsilon$
 $\langle \text{Power} \rangle \rightarrow \langle \text{Unary} \rangle \langle \text{Power}' \rangle$
 $\langle \text{Power}' \rangle \rightarrow ^ \langle \text{Power} \rangle \mid \varepsilon$
 $\langle \text{Unary} \rangle \rightarrow + \langle \text{Unary} \rangle$
 $\quad \mid - \langle \text{Unary} \rangle$
 $\quad \mid \text{not } \langle \text{Unary} \rangle$
 $\quad \mid \langle \text{Factor} \rangle$
 $\langle \text{Factor} \rangle \rightarrow \text{id}$
 $\quad \mid \text{integer}$
 $\quad \mid \text{float}$
 $\quad \mid \text{string}$
 $\quad \mid \text{true}$
 $\quad \mid \text{false}$
 $\quad \mid (\langle \text{Expr} \rangle)$

The transformed grammar removes left recursion from arithmetic and Boolean expressions. It also separates expression levels to enforce operator precedence. The lowest precedence is handled by or, followed by and, relational operators, addition and subtraction, multiplication/division/modulo, power, and finally unary operators. The power operator is written as right-associative using the rule $\langle \text{Power} \rangle \rightarrow ^ \langle \text{Power} \rangle | \epsilon$.

2.4 FIRST Sets

$\text{FIRST}(\langle \text{Program} \rangle) = \{ \text{start} \}$

$\text{FIRST}(\langle \text{StmtList} \rangle) = \{ \text{var}, \text{id}, \text{print}, \text{read}, \text{if}, \text{while}, \text{func}, \text{do}, \epsilon \}$

$\text{FIRST}(\langle \text{Stmt} \rangle) = \{ \text{var}, \text{id}, \text{print}, \text{read}, \text{if}, \text{while}, \text{func}, \text{do} \}$

$\text{FIRST}(\langle \text{VarDecl} \rangle) = \{ \text{var} \}$

$\text{FIRST}(\langle \text{Type} \rangle) = \{ \text{int}, \text{float}, \text{string}, \text{bool} \}$

$\text{FIRST}(\langle \text{AssignStmt} \rangle) = \{ \text{id} \}$

$\text{FIRST}(\langle \text{PrintStmt} \rangle) = \{ \text{print} \}$

$\text{FIRST}(\langle \text{ReadStmt} \rangle) = \{ \text{read} \}$

$\text{FIRST}(\langle \text{IfStmt} \rangle) = \{ \text{if} \}$

$\text{FIRST}(\langle \text{ElsePart} \rangle) = \{ \text{else}, \epsilon \}$

$\text{FIRST}(\langle \text{WhileStmt} \rangle) = \{ \text{while} \}$

$\text{FIRST}(\langle \text{FuncDecl} \rangle) = \{ \text{func} \}$

$\text{FIRST}(\langle \text{ReturnType} \rangle) = \{ \text{int}, \text{float}, \text{string}, \text{bool}, \text{void} \}$

$\text{FIRST}(\langle \text{ParamList} \rangle) = \{ \text{int}, \text{float}, \text{string}, \text{bool}, \epsilon \}$

$\text{FIRST}(\langle \text{ParamRest} \rangle) = \{ ,, \epsilon \}$

$\text{FIRST}(\langle \text{Param} \rangle) = \{ \text{int}, \text{float}, \text{string}, \text{bool} \}$

$\text{FIRST}(\langle \text{FuncCall} \rangle) = \{ \text{do} \}$
 $\text{FIRST}(\langle \text{ArgList} \rangle) = \{ \text{id, integer, float, string, true, false, (, +, -, not, } \}$
 $\text{FIRST}(\langle \text{ArgRest} \rangle) = \{ ,, \epsilon \}$
 $\text{FIRST}(\langle \text{BoolExpr} \rangle) = \{ \text{id, integer, float, string, true, false, (, +, -, not} \}$
 $\text{FIRST}(\langle \text{BoolExpr}' \rangle) = \{ \text{or, } \epsilon \}$
 $\text{FIRST}(\langle \text{BoolTerm} \rangle) = \{ \text{id, integer, float, string, true, false, (, +, -, not} \}$
 $\text{FIRST}(\langle \text{BoolTerm}' \rangle) = \{ \text{and, } \epsilon \}$
 $\text{FIRST}(\langle \text{BoolFactor} \rangle) = \{ \text{not, id, integer, float, string, true, false, (, +, -} \}$
 $\text{FIRST}(\langle \text{RelPart} \rangle) = \{ ==, !=, <, >, <=, >=, \epsilon \}$
 $\text{FIRST}(\langle \text{RelOp} \rangle) = \{ ==, !=, <, >, <=, >= \}$
 $\text{FIRST}(\langle \text{Expr} \rangle) = \{ \text{id, integer, float, string, true, false, (, +, -, not} \}$
 $\text{FIRST}(\langle \text{Expr}' \rangle) = \{ +, -, \epsilon \}$
 $\text{FIRST}(\langle \text{Term} \rangle) = \{ \text{id, integer, float, string, true, false, (, +, -, not} \}$
 $\text{FIRST}(\langle \text{Term}' \rangle) = \{ *, /, \%, \epsilon \}$
 $\text{FIRST}(\langle \text{Power} \rangle) = \{ \text{id, integer, float, string, true, false, (, +, -, not} \}$
 $\text{FIRST}(\langle \text{Power}' \rangle) = \{ ^, \epsilon \}$
 $\text{FIRST}(\langle \text{Unary} \rangle) = \{ +, -, \text{not, id, integer, float, string, true, false, (} \}$
 $\text{FIRST}(\langle \text{Factor} \rangle) = \{ \text{id, integer, float, string, true, false, (} \}$

2.5 FOLLOW Sets

$\text{FOLLOW}(\langle \text{Program} \rangle) = \{ \$ \}$

```

FOLLOW(<StmtList>) = { finish, else, $ }

FOLLOW(<Stmt>) = { var, id, print, read, if, while, func,
do, finish, else, $ }

FOLLOW(<VarDecl>) = { var, id, print, read, if, while,
func, do, finish, else, $ }

FOLLOW(<Type>) = { id }

FOLLOW(<AssignStmt>) = { var, id, print, read, if, while,
func, do, finish, else, $ }

FOLLOW(<PrintStmt>) = { var, id, print, read, if, while,
func, do, finish, else, $ }

FOLLOW(<ReadStmt>) = { var, id, print, read, if, while,
func, do, finish, else, $ }

FOLLOW(<IfStmt>) = { var, id, print, read, if, while,
func, do, finish, else, $ }

FOLLOW(<ElsePart>) = { finish }

FOLLOW(<WhileStmt>) = { var, id, print, read, if, while,
func, do, finish, else, $ }

FOLLOW(<FuncDecl>) = { var, id, print, read, if, while,
func, do, finish, else, $ }

FOLLOW(<ReturnType>) = { id }

FOLLOW(<ParamList>) = { ) }

FOLLOW(<ParamRest>) = { ) }

FOLLOW(<Param>) = { ,, ) }

FOLLOW(<FuncCall>) = { var, id, print, read, if, while,
func, do, finish, else, $ }

FOLLOW(<ArgList>) = { ) }

FOLLOW(<ArgRest>) = { ) }

FOLLOW(<BoolExpr>) = { then, do, ) }

FOLLOW(<BoolExpr'>) = { then, do, ) }

```

```

FOLLOW(<BoolTerm>) = { or, then, do, ) }

FOLLOW(<BoolTerm'>) = { or, then, do, ) }

FOLLOW(<BoolFactor>) = { and, or, then, do, ) }

FOLLOW(<RelPart>) = { and, or, then, do, ) }

FOLLOW(<RelOp>) = { id, integer, float, string, true,
false, (, +, -, not }

FOLLOW(<Expr>) = { ;, ), ,, ==, !=, <, >, <=, >=, and,
or, then, do }

FOLLOW(<Expr'>) = { ;, ), ,, ==, !=, <, >, <=, >=, and,
or, then, do }

FOLLOW(<Term>) = { +, -, ;, ), ,, ==, !=, <, >, <=, >=,
and, or, then, do }

FOLLOW(<Term'>) = { +, -, ;, ), ,, ==, !=, <, >, <=, >=,
and, or, then, do }

FOLLOW(<Power>) = { *, /, %, +, -, ;, ), ,, ==, !=, <, >,
<=, >=, and, or, then, do }

FOLLOW(<Power'>) = { *, /, %, +, -, ;, ), ,, ==, !=, <,
>, <=, >=, and, or, then, do }

FOLLOW(<Unary>) = { ^, *, /, %, +, -, ;, ), ,, ==, !=, <,
>, <=, >=, and, or, then, do }

FOLLOW(<Factor>) = { ^, *, /, %, +, -, ;, ), ,, ==, !=,
<, >, <=, >=, and, or, then, do }

```

The FOLLOW sets represent the valid symbols that can appear immediately after each non-terminal during parsing. These sets are essential for constructing the LL(1) parsing table and ensuring that the parser can make correct predictive decisions without ambiguity.

2.6 LL(1) Parsing Table

The LL(1) parsing table is constructed using the FIRST and FOLLOW sets of the transformed grammar. Each table entry contains the production rule that should be applied when a specific non-terminal appears on the stack and a specific terminal appears in the input stream. The table allows the parser to make predictive parsing decisions without backtracking, ensuring that the grammar satisfies the LL(1) property.

Table 1: Partial LL(1) Parsing Table

Non-Terminal	Input Symbol	Production Rule
<Program>	start	<Program> $\rightarrow$ start <StmtList> finish
<StmtList>	var	<StmtList> $\rightarrow$ <Stmt> <StmtList>
<StmtList>	id	<StmtList> $\rightarrow$ <Stmt> <StmtList>
<StmtList>	print	<StmtList> $\rightarrow$ <Stmt> <StmtList>
<StmtList>	read	<StmtList> $\rightarrow$ <Stmt> <StmtList>
<StmtList>	if	<StmtList> $\rightarrow$ <Stmt> <StmtList>
<StmtList>	while	<StmtList> $\rightarrow$ <Stmt> <StmtList>
<StmtList>	func	<StmtList> $\rightarrow$ <Stmt> <StmtList>
<StmtList>	do	<StmtList> $\rightarrow$ <Stmt> <StmtList>
<StmtList>	finish	<StmtList> $\rightarrow \epsilon$
<Stmt>	var	<Stmt> $\rightarrow$ <VarDecl>
<Stmt>	id	<Stmt> $\rightarrow$ <AssignStmt>

<Stmt>	print	<Stmt> → <PrintStmt>
<Stmt>	read	<Stmt> → <ReadStmt>
<Stmt>	if	<Stmt> → <IfStmt>
<Stmt>	while	<Stmt> → <WhileStmt>
<Stmt>	func	<Stmt> → <FuncDecl>
<Stmt>	do	<Stmt> → <FuncCall>
<VarDecl>	var	<VarDecl> → var <Type> id ;
<AssignStmt>	id	<AssignStmt> → id = <Expr> ;
<PrintStmt>	print	<PrintStmt> → print <Expr> ;
<ReadStmt>	read	<ReadStmt> → read id ;
<IfStmt>	if	<IfStmt> → if <BoolExpr> then <StmtList> <ElsePart> finish
<WhileStmt>	while	<WhileStmt> → while <BoolExpr> do <StmtList> finish
<FuncCall>	do	<FuncCall> → do id (<ArgList>) ;
<Expr>	id	<Expr> → <Term> <Expr'>
<Expr>	integer	<Expr> → <Term> <Expr'>
<Expr>	float	<Expr> → <Term> <Expr'>
<Expr>	string	<Expr> → <Term> <Expr'>
<Expr>	(	<Expr> → <Term> <Expr'>
<Expr'>	+	<Expr'> → + <Term> <Expr'>
<Expr'>	-	<Expr'> → - <Term> <Expr'>
<Expr'>	;	<Expr'> → ε
<Term'>	*	<Term'> → * <Power> <Term'>

<Term'>	/	<Term'> → / <Power> <Term'>
<Term'>	%	<Term'> → % <Power> <Term'>
<Term'>	+	<Term'> → ε
<Term'>	-	<Term'> → ε
<Term'>	;	<Term'> → ε

The parsing table demonstrates how predictive parsing decisions are made using one lookahead token. Each entry maps a non-terminal and an input symbol to the correct production rule. Empty entries in the table indicate syntax errors.

2.7 Parser Algorithm

The parser in HealthScript is implemented using the Recursive Descent Parsing technique, which is a top-down parsing approach suitable for LL(1) grammars. In this method, each non-terminal in the grammar is represented by a separate parsing function. The parser processes the token stream generated by the lexical analyzer and recursively applies grammar rules to verify the correctness of the program structure. The parsing process begins with the <Program> rule, which expects the program to start with the keyword start and end with finish. The parser then recursively processes statement lists, declarations, expressions, control structures, and function-related constructs according to the transformed LL(1) grammar. The parser uses a lookahead token mechanism to determine which production rule should be applied. At each step, the parser compares the current token with the expected token. If the token matches, the parser advances to the next token. Otherwise, a syntax error is reported.

The recursive descent parser includes parsing functions for:

- Program structure
- Statement lists
- Variable declarations
- Assignment statements
- Conditional statements
- Loop statements
- Function declarations and calls
- Arithmetic and Boolean expressions

Operator precedence is handled by separating expression parsing into multiple levels, including expressions, terms, powers, unary operations, and factors. This ensures correct parsing of arithmetic operations according to precedence and associativity rules. Error handling is also integrated into the parser. Whenever an unexpected token is encountered, the parser generates a descriptive syntax error message indicating the expected token, the received token, and the corresponding line number. The parser may also attempt simple recovery strategies to continue parsing the remaining input. Overall, the parser validates the syntactic correctness of HealthScript programs and provides structured error reporting, forming the second major phase of the compiler front-end.

2.8 Recursive Descent Parsing Procedure

1. Initialize the parser with the token stream generated by the scanner.
2. Set the current token as the first token in the stream.
3. Begin parsing using the Program() function.

4. For each non-terminal:

- Check the current token.
- Select the appropriate production rule.
- Match expected terminals.
- Recursively call parsing functions for non-terminals.

5. If the current token matches the expected token:

- Advance to the next token.

6. Otherwise:

- Generate a syntax error message.

7. Continue parsing until:

- The entire token stream is processed successfully, or
- A fatal syntax error is encountered.

8. If parsing completes successfully and the end-of-file symbol is reached:

- Accept the program as syntactically correct.

2.9 Parser Design Note

The parser was designed according to the LL(1) transformed grammar to ensure deterministic parsing without backtracking. Recursive descent parsing was selected because it is simple to implement, easy to debug, and well suited for educational compiler projects.

2.10 Parse Tree Representation

The parser generates a parse tree to represent the hierarchical syntactic structure of the input program according to the grammar rules of HealthScript. The parse tree visually demonstrates how tokens are derived from the grammar productions and how expressions and statements are organized within the program. Each internal node in the parse tree represents a non-terminal symbol, while leaf nodes represent terminal symbols (tokens). The parse tree helps verify that the parser correctly applies production rules and respects operator precedence and associativity. The parse tree also assists in debugging and validating the correctness of the parser implementation.

Note

The parser generates and displays the parse tree dynamically during execution, as shown in the test case screenshots.

Example Parse Tree

Example Input Program:

```
start  
  
var int x;  
  
x = 5 + 3;  
  
finish
```

(Consoles)

<Program>

├─ start

├─ <StmtList>

| └─ <Stmt>

```

|   |   └─ <VarDecl>
|   |       └─ var
|   |       └─ int
|   |       └─ id(x)
|   |       └─ ;
|   └─ <StmtList>
|   |   └─ <Stmt>
|   |   |   └─ <AssignStmt>
|   |   |   |   └─ id(x)
|   |   |   |   └─ =
|   |   |   |   └─ <Expr>
|   |   |   |   |   └─ <Term>
|   |   |   |   |   |   └─ integer(5)
|   |   |   |   |   └─ +
|   |   |   |   |   └─ <Term>
|   |   |   |   |       └─ integer(3)
|   |   |   |   └─ ;
|   |   └─ ε
└─ finish

```

The parse tree above illustrates how the parser derives the input program according to the LL(1) grammar rules. It demonstrates the hierarchical relationship between declarations, assignments, and arithmetic expressions.

2.11 Syntax Error Handling

The HealthScript parser includes a syntax error handling mechanism to detect invalid program structures during parsing. Whenever the parser encounters an unexpected

token that does not match the expected grammar rule, it generates a descriptive syntax error message.

The error message includes:

- The line number where the error occurred
- The unexpected token received
- The expected token or grammar construct
- Syntax error messages include the line number, the expected token, and the received token.

This helps users identify and correct syntax mistakes efficiently.

Examples of syntax errors detected by the parser include:

- Missing semicolons
- Missing finish keyword
- Invalid statement structure
- Unmatched parentheses
- Invalid expressions
- Incorrect function declarations
- Missing then or do keywords

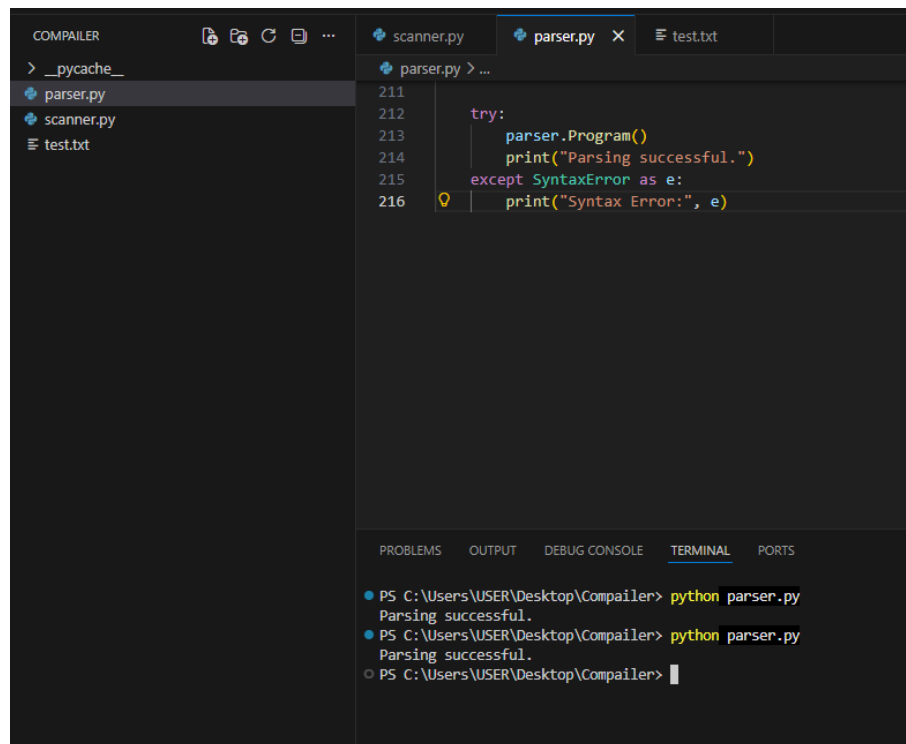
The parser may also apply simple recovery techniques, such as skipping invalid tokens or continuing parsing from the next statement, in order to report multiple syntax errors within the same program.

2.12 Parser Test Cases

1. Test Case 1: Valid Program

Code

```
start  
  
var int x;  
  
x = 5 + 3;  
  
print x;  
  
finish
```



Output

Parsing successful.

No syntax errors found.

2. Test Case 2: Valid Conditional Statement

Code

```
start
```



```
var int temp;

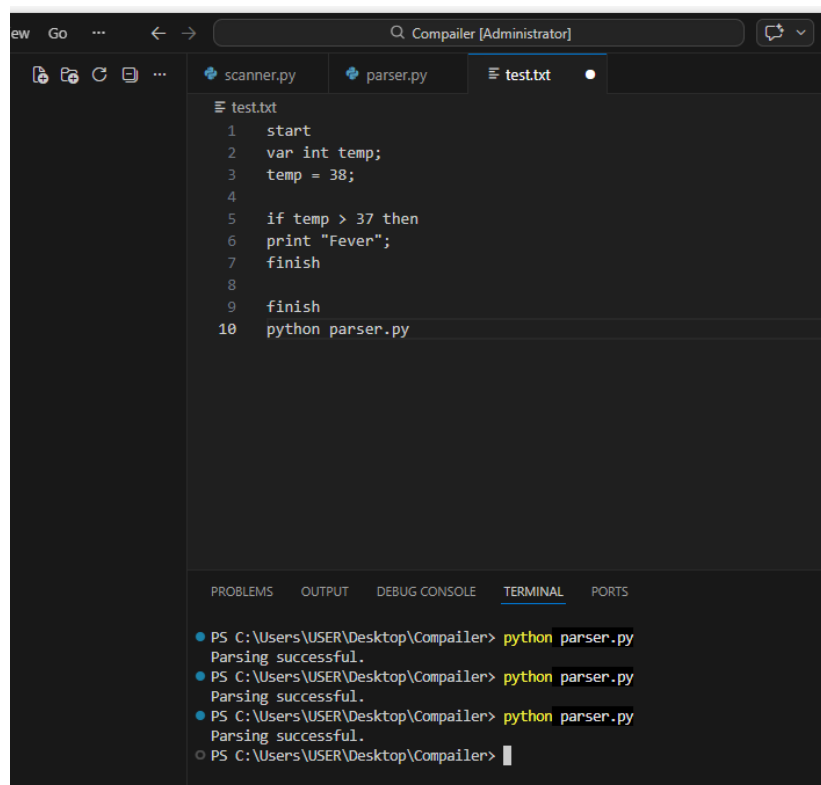
temp = 38;

f temp > 37 then

print "Fever";

finish

finish
```



The screenshot shows a Visual Studio Code editor window titled "Compiler [Administrator]". The editor has three tabs: "scanner.py", "parser.py", and "test.txt". The "test.txt" tab is active, displaying the following code:

```
1 start
2 var int temp;
3 temp = 38;
4
5 if temp > 37 then
6   print "Fever";
7   finish
8
9 finish
10 python parser.py
```

Below the editor, the "TERMINAL" panel is open, showing the output of running the script:

```
PS C:\Users\USER\Desktop\Compiler> python parser.py
Parsing successful.
PS C:\Users\USER\Desktop\Compiler> python parser.py
Parsing successful.
PS C:\Users\USER\Desktop\Compiler> python parser.py
Parsing successful.
PS C:\Users\USER\Desktop\Compiler>
```

Output

Parsing successful.

No syntax errors found.

3. Test Case 3: Valid Loop Statement

Code

```
start
```

```
var int i;

i = 0;

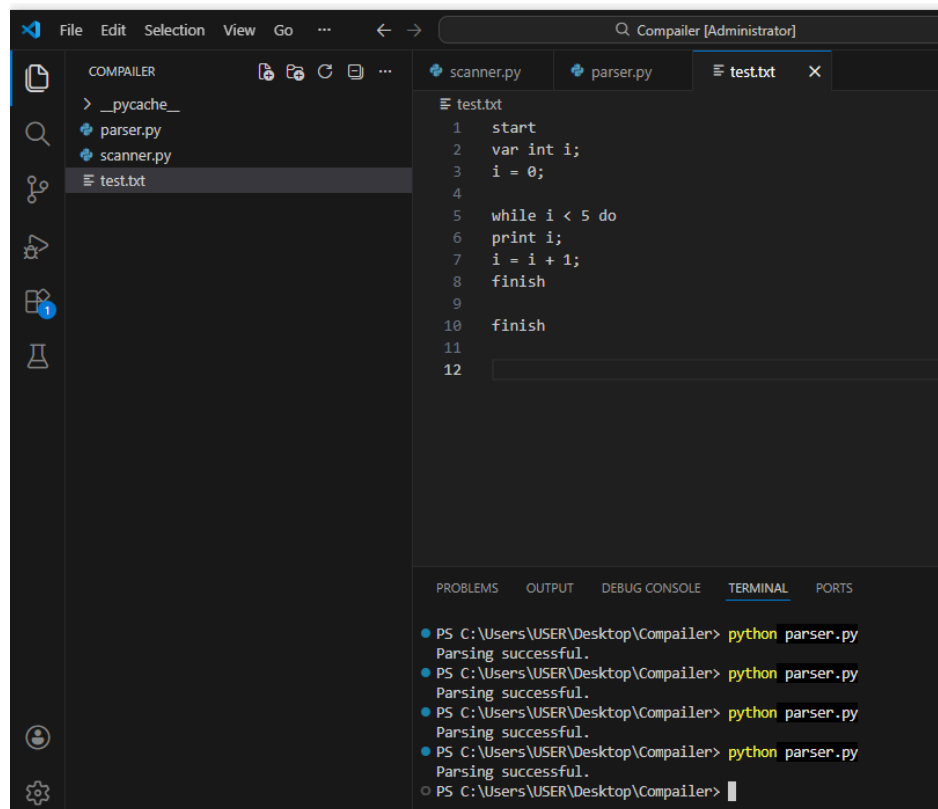
while i < 5 do

print i;

i = i + 1;

finish

finish
```



Output

Parsing successful.

No syntax errors found.

4. Test Case 4: Missing Semicolon

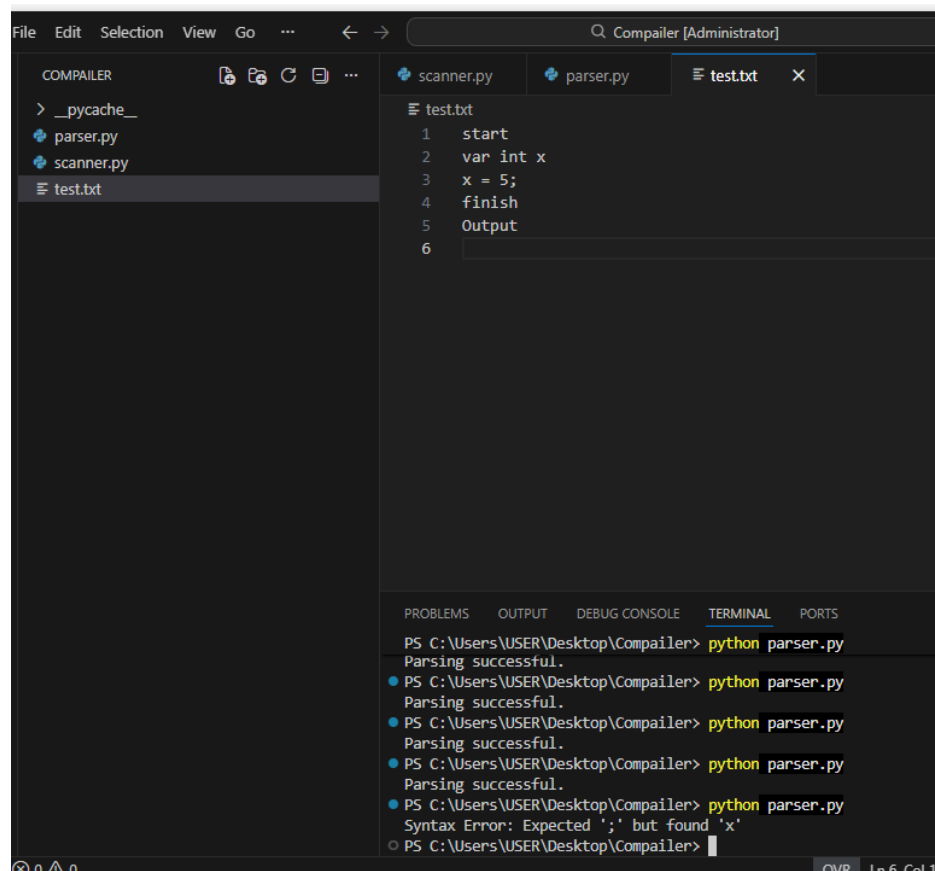
Code

start

var int x

x = 5;

finish



Output

Syntax Error: Expected ';' but found 'x'

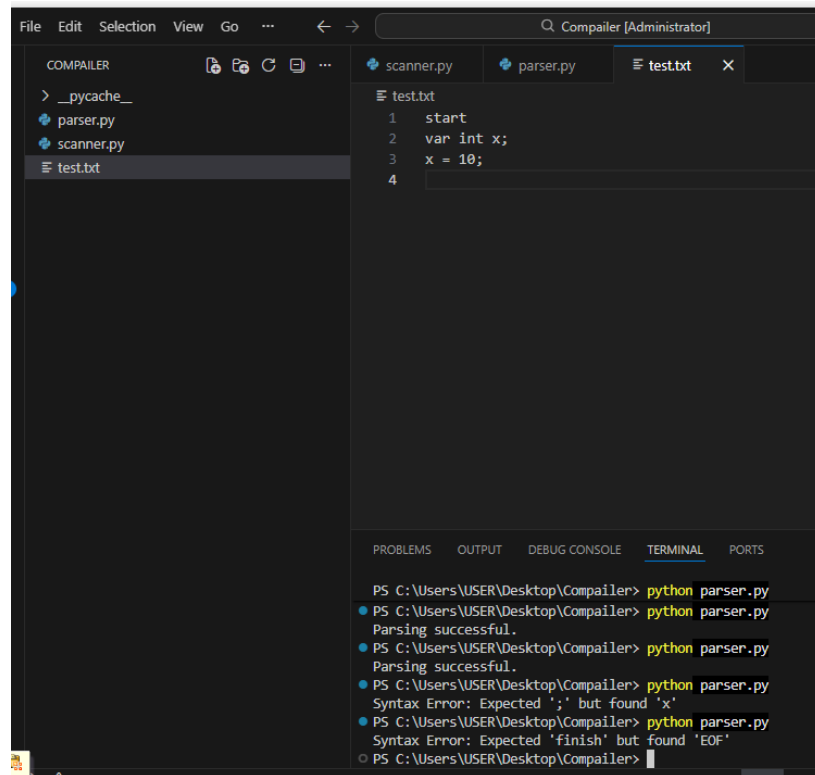
5. Test Case 5: Missing finish Keyword

Code

start

```
var int x;
```

```
x = 10;
```



Output

Syntax Error:

Expected 'finish' at end of program.

The test cases demonstrate the ability of the parser to correctly validate valid HealthScript programs and detect syntax errors in invalid programs. Both successful parsing and syntax error reporting were verified during testing.

6. Test Case 6 : Function Declaration and Function Call

Code

```
start
```

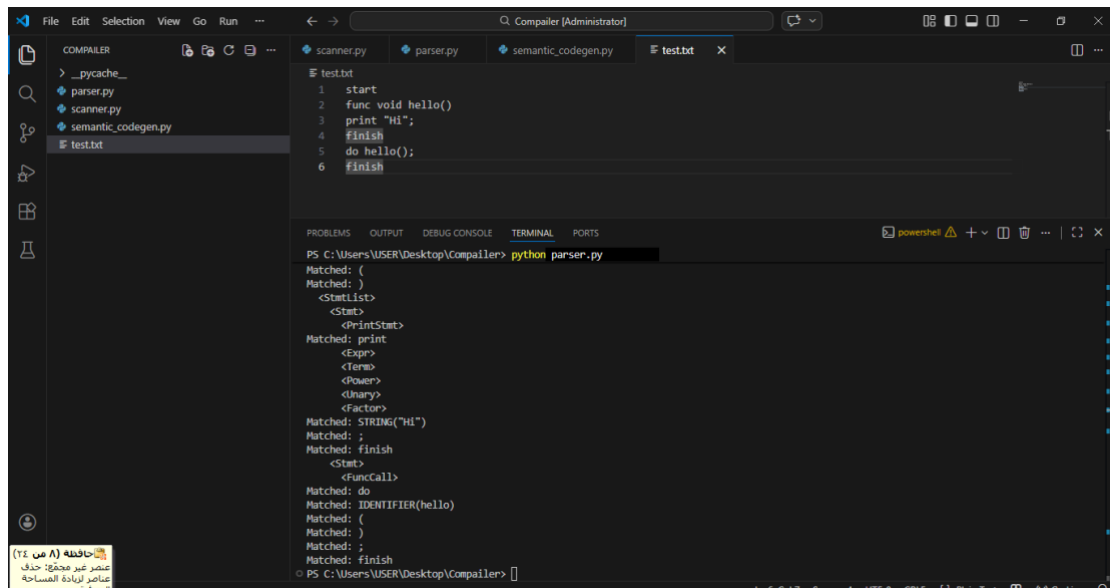
```
func void hello()
```

```
print "Hi";
```

```
finish
```

```
do hello();
```

```
finish
```



The screenshot shows the 'Compailer' IDE interface. The editor window displays a file named 'test.txt' with the following code:

```
1 start
2 func void hello()
3 print "Hi";
4 finish
5 do hello();
6 finish
```

The terminal window at the bottom shows the command 'python parser.py' being executed. The output displays a parse tree structure for the input code:

```
Matched: (
Matched: )
<StatList>
<Stat>
  <PrintStat>
    Matched: print
    Matched: <Expr>
      Matched: <Term>
        Matched: <Power>
          Matched: <Unary>
            Matched: <Factor>
              Matched: STRING("Hi")
            Matched: ;
          Matched: finish
        Matched: <Stat>
          Matched: <FuncCall>
            Matched: do
            Matched: IDENTIFIER(hello)
            Matched: (
            Matched: ;
            Matched: finish
          Matched: finish
        Matched: finish
      Matched: finish
    Matched: finish
  Matched: finish
Matched: finish
```

Output:

Parsing successful.

The parser successfully recognized the function declaration, the print statement inside the function body, and the function call statement. The parse tree was generated and displayed in the terminal, showing the matched tokens and grammar structure for the function declaration and function call.

7. Test Case 7 : for and / or / not

```
start
```

```
var int x;
```

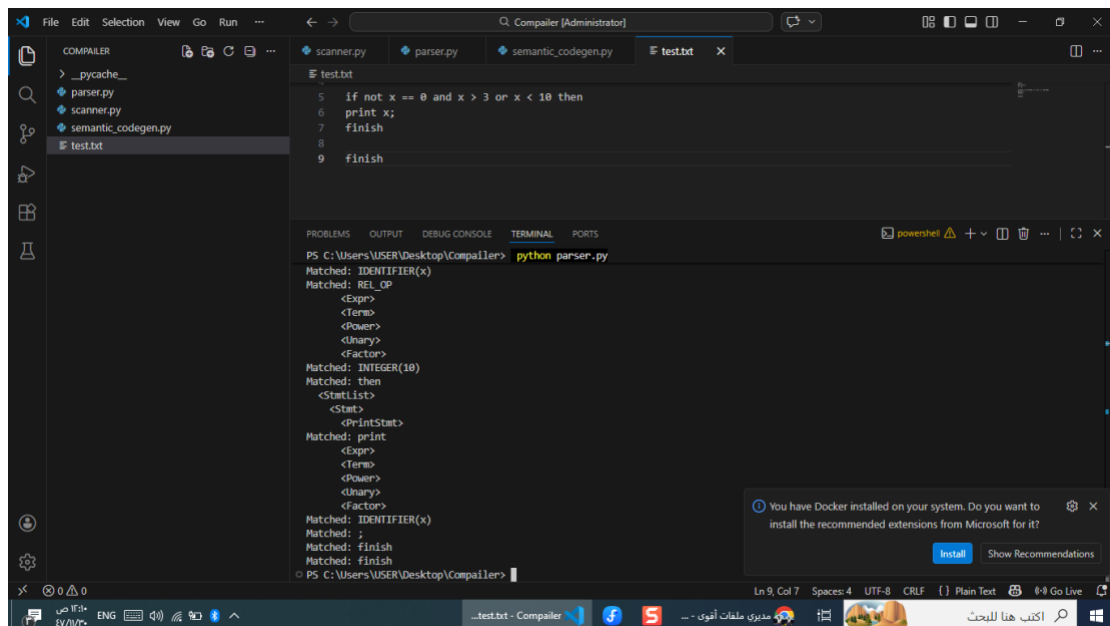
```
x = 5;
```

if not x == 0 and x > 3 or x < 10 then

print x;

finish

finish



output

Parsing successful.

Parse Tree Display

8. Case Study 8 : line number

Code

start

var int x

x = 5;

finish

```
COMPILER
> __pycache__
  parser.py
  scanner.py
  semantic_codegen.py
  test.txt

test.txt
1 start
2 var int x
3 x = 5;
4 finish

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\USER\Desktop\Compiler> python parser.py
<Expr>
  <Term>
    <Power>
      <Unary>
        <Factor>
          Matched: INTEGER(10)
        Matched: then
      <StatList>
        <Stat>
          <PrintStat>
            Matched: print
          <Expr>
            <Term>
              <Power>
                <Unary>
                  <Factor>
                    Matched: IDENTIFIER(x)
                  Matched: ;
                Matched: finish
              Matched: finish
            Matched: finish
          PS C:\Users\USER\Desktop\Compiler> python parser.py
          Syntax Error at line 3: Expected ';' but found 'x'
          PS C:\Users\USER\Desktop\Compiler>

You have Docker installed on your system. Do you want to
install the recommended extensions from Microsoft for it?
[Install] [Show Recommendations]
```

Output

Syntax Error at line ...

2.13 Conclusion

In this phase, a complete syntax analyzer for the HealthScript language was successfully designed and implemented. A Context-Free Grammar (CFG) was created to define the structure of the language, including declarations, assignments, expressions, control structures, functions, and input/output statements. The grammar was transformed into an LL(1)-compatible form by eliminating left recursion and applying grammar transformations suitable for predictive parsing.

FIRST and FOLLOW sets were constructed, and a partial LL(1) parsing table was generated to support parsing decisions. A recursive descent parser was designed to validate token streams produced by the lexical analyzer from Phase 1. The parser correctly recognized syntactically valid programs and generated meaningful syntax error messages for invalid inputs.

The implementation demonstrates the fundamental principles of syntax analysis and predictive parsing in compiler construction. Overall, the syntax analysis phase provides a strong foundation for the next compiler stages, including semantic analysis and intermediate code generation in Phase 3.

Chapter 3: Semantic Analysis and Code Generation

3.0 Introduction

Semantic analysis is the third phase of the HealthScript compiler front-end. After the source code successfully passes lexical and syntax analysis, the semantic analyzer verifies whether the program is logically correct according to the semantic rules of the language. This phase ensures that variables are declared before use, assignments are type-compatible, duplicate declarations are prevented, and arithmetic operations are performed correctly.

In addition to semantic validation, this phase introduces intermediate code generation using Three-Address Code (TAC). TAC provides a simplified internal representation of the program, making expressions and operations easier to analyze and execute during later compilation stages.

The semantic analyzer also utilizes an enhanced symbol table that stores additional information about identifiers, including variable types, declaration lines, and scope levels. This information supports semantic verification and improves program analysis.

This chapter presents the semantic analysis objective, enhanced symbol table structure, semantic rules, semantic error handling, TAC generation approach, test cases, and the complete end-to-end compilation workflow of the HealthScript compiler.

3.1 Phase 3 Objective

The objective of Phase 3 is to extend the HealthScript compiler front-end by adding semantic analysis and code generation. The semantic analyzer checks whether the

program is logically meaningful after passing lexical and syntax analysis. It verifies declarations, duplicate variables, type compatibility, and semantic errors. After successful semantic analysis, the system generates Three-Address Code (TAC) as an intermediate representation.

3.2 Enhanced Symbol Table

In Phase 3, the symbol table was extended to store additional information about identifiers. Instead of only storing the identifier name and first occurrence line, the enhanced symbol table stores the identifier name, data type, scope level, and declaration line. The enhanced symbol table supports semantic checking by allowing the compiler to verify whether variables are declared before use, whether duplicate declarations exist, and whether assignments match the declared variable types.

Field	Description
Identifier Name	Name of the variable
Type	int, float, string, bool
Scope	global
Declaration Line	Line where identifier was declared

3.3 Semantic Rules

The semantic analyzer applies the following rules:

1. Every variable must be declared before use.
2. A variable cannot be declared more than once in the same scope.
3. The assigned value must be compatible with the declared variable type.

4. Integer values can be assigned to float variables.
5. String values cannot be assigned to int or float variables.
6. Arithmetic operations are only allowed between numeric types.
7. Division by zero is reported when the divisor is a literal zero.
8. Print statements must reference valid declared identifiers or valid literals.

3.4 Semantic Error Handling

The semantic analyzer reports errors with clear messages and line numbers. Detected semantic errors include undeclared variables, duplicate declarations, incompatible assignments, invalid arithmetic operations, and division by zero.

3.5 Code Generation Approach

For code generation, the project uses Three-Address Code (TAC). TAC was selected because it is simple, readable, and suitable for educational compiler projects. Each TAC instruction contains at most one operation, which makes it easy to analyze and interpret.

3.6 Test Cases

1. Test Case 1: Correct Program with TAC Generation

Code

```
start
var int x;
x = 5 + 3;
print x;
finish
```

Output

Parsing successful.

Semantic analysis completed successfully.

Generated Three-Address Code (TAC):

t1 = 5 + 3

x = t1

print x

End-to-end compilation completed successfully.

2. Test Case 2: Type Mismatch Error

Code:

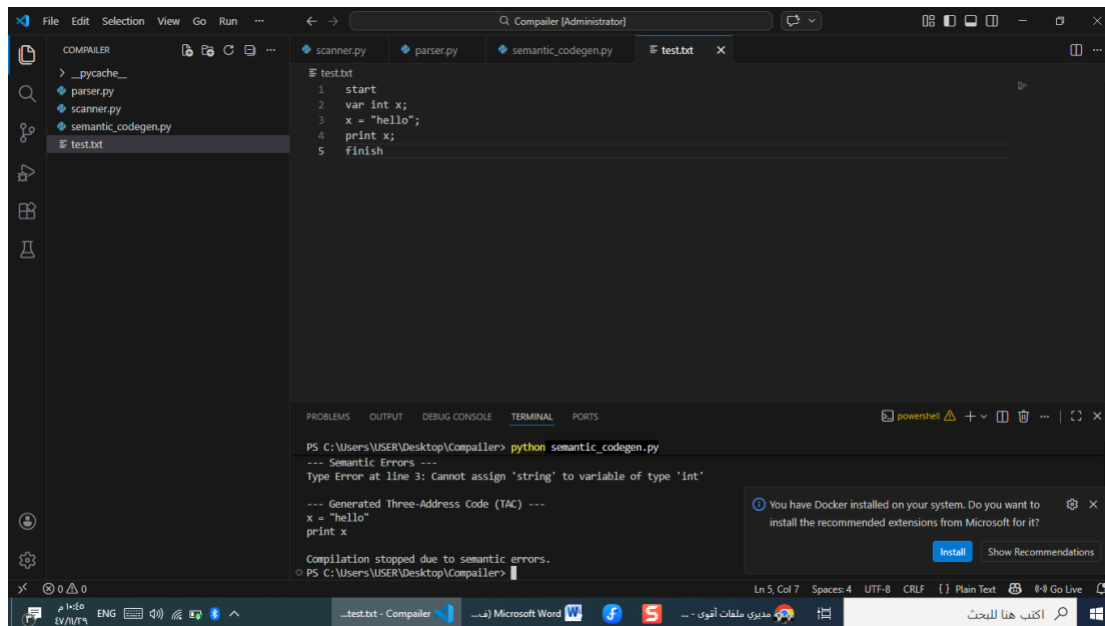
start

var int x;

x = "hello";

print x;

finish



Output:

Type Error at line 3: Cannot assign 'string' to variable of type 'int'

Compilation stopped due to semantic errors.

3. Test Case 3: Undeclared Variable Error

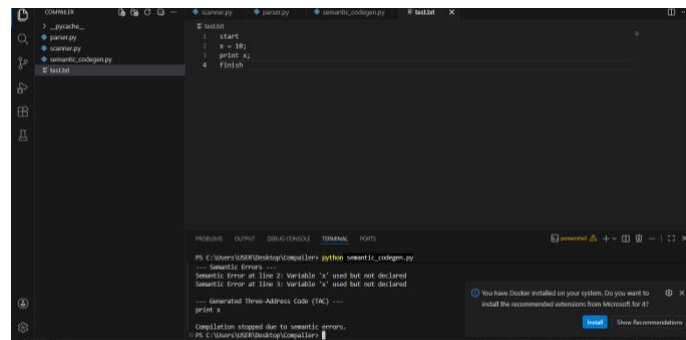
Code:

start

x = 10;

print x;

finish



Output:

Semantic Error at line 2: Variable 'x' used but not declared

Semantic Error at line 3: Variable 'x' used but not declared

4. Test Case 4 — Duplicate Declaration Error

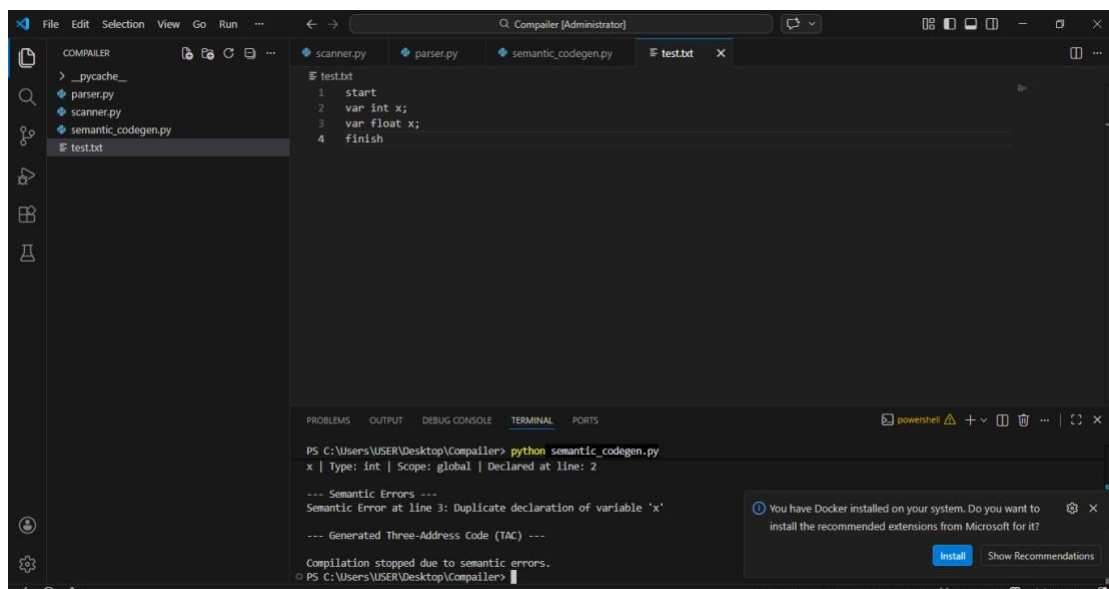
Code:

start

var int x;

var float x;

finish



Output:

Semantic Error at line 3: Duplicate declaration of variable 'x'

Compilation stopped due to semantic errors.

5. Test Case 5 — Correct Program with Float Operations

Code:

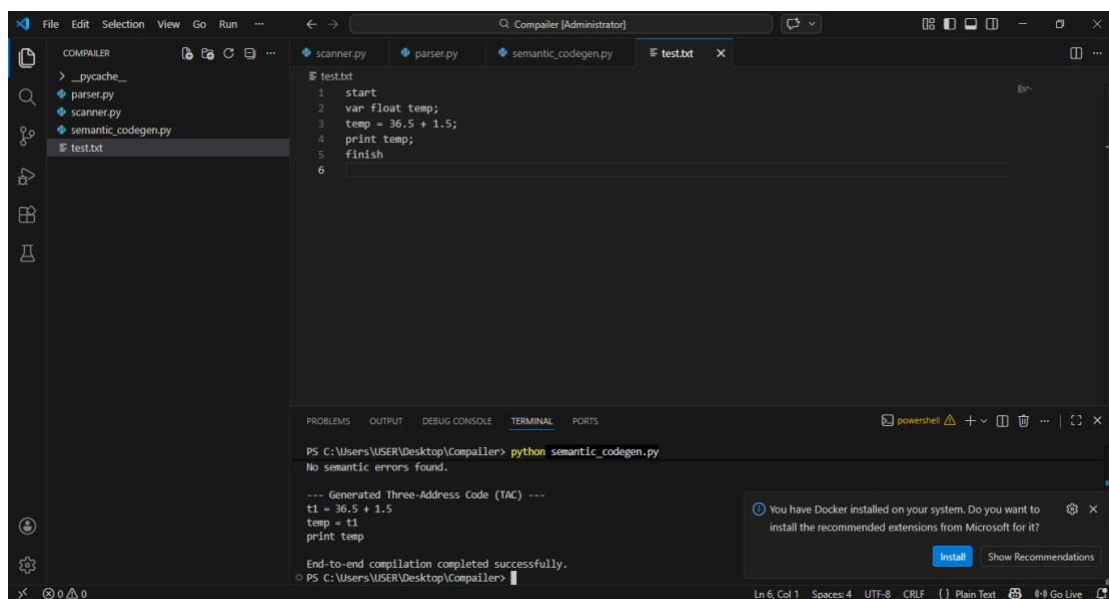
start

var float temp;

temp = 36.5 + 1.5;

print temp;

finish



Output:

No semantic errors found.

Generated Three-Address Code (TAC):

t1 = 36.5 + 1.5

temp = t1

print temp

End-to-end compilation completed successfully.

6. Test Case 6: Division by Zero Error

Code:

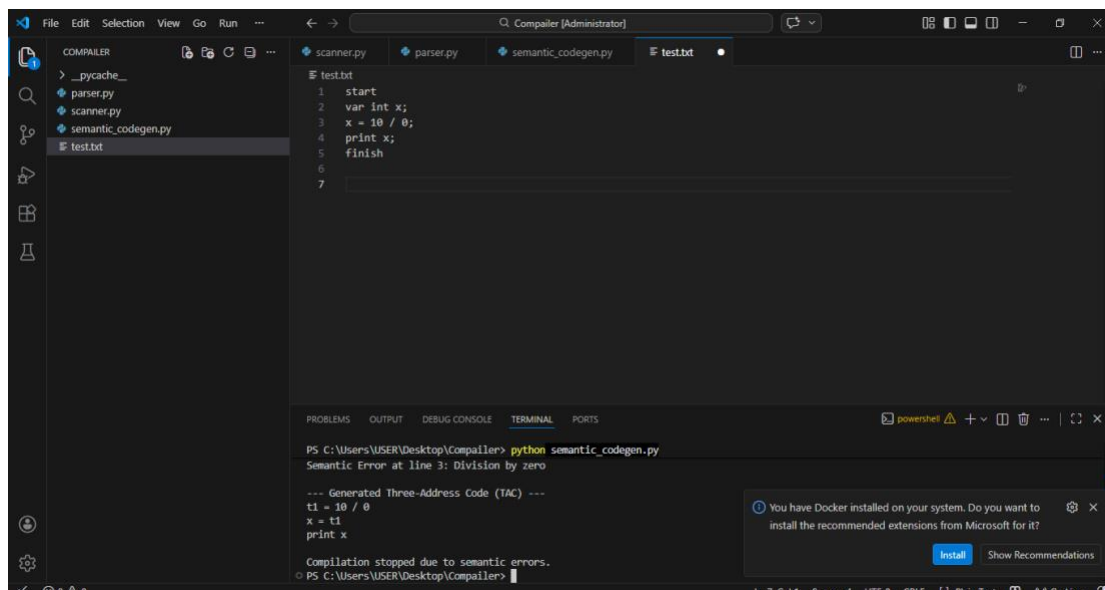
start

var int x;

x = 10 / 0;

print x;

finish



Output:

Semantic Error at line 3: Division by zero

Compilation stopped due to semantic errors.

3.7 End-to-End Compilation

The HealthScript compiler demonstrates a complete end-to-end compilation pipeline by processing source code through all compiler front-end phases. The process begins with lexical analysis, where the scanner reads the source program and converts it into a sequence of tokens while detecting lexical errors such as invalid symbols or malformed identifiers.

After lexical analysis, the generated token stream is passed to the syntax analyzer (parser). The parser verifies whether the token sequence conforms to the Context-Free Grammar (CFG) rules defined for the HealthScript language. During this phase, the parser checks the correctness of declarations, statements, expressions, and control structures using the Recursive Descent Parsing technique.

Once syntax analysis is completed successfully, the semantic analyzer validates the logical meaning of the program. It performs several semantic checks including variable declaration verification, duplicate declaration detection, type compatibility checking, arithmetic validation, and division-by-zero detection. The semantic analyzer also uses the enhanced symbol table to manage identifier information such as variable types, declaration lines, and scope levels.

If no semantic errors are found, the compiler proceeds to the code generation phase. In this project, Three-Address Code (TAC) is generated as an intermediate representation of the program. TAC simplifies expressions into smaller instructions that are easier to analyze and execute.

The complete compilation workflow of the HealthScript compiler is summarized as follows:

Source Code → Lexical Analysis → Syntax Analysis → Semantic Analysis → TAC Generation

The end-to-end compilation process was tested using multiple correct and incorrect HealthScript programs. Correct programs successfully generated TAC instructions, while incorrect programs produced clear lexical, syntax, or semantic error messages with line numbers.

3.8 Conclusion

Phase 3 successfully extends the HealthScript compiler by adding semantic analysis and intermediate code generation. The semantic analyzer validates program meaning by checking declarations, type compatibility, duplicate identifiers, undeclared variables, and division-by-zero errors. The enhanced symbol table supports semantic verification and improves identifier management throughout the compilation process. In addition, the compiler generates readable Three-Address Code (TAC) as an intermediate representation for valid programs. The successful integration of lexical analysis, syntax analysis, semantic analysis, and TAC generation demonstrates a complete compiler front-end workflow for the HealthScript language.

Chapter 4: Overall Conclusion

4.0 Introduction

This chapter presents the overall conclusion of the HealthScript compiler project. It summarizes the work completed throughout all project phases, including lexical analysis, syntax analysis, semantic analysis, and intermediate code generation. The chapter also highlights the achievements of the project, the compiler capabilities, and possible future improvements.

4.1 Project Summary

The HealthScript compiler project successfully implemented a complete compiler front-end using Python. The project was divided into three major phases. Phase 1 focused on lexical analysis, where the scanner transformed source code into tokens and detected lexical errors. Phase 2 implemented syntax analysis using a recursive descent parser based on an LL(1)-transformed grammar. Phase 3 extended the compiler by adding semantic analysis and Three-Address Code (TAC) generation.

The compiler supports variable declarations, assignments, arithmetic expressions, Boolean expressions, conditional statements, loops, function declarations, and function calls. In addition, the system provides structured error handling and meaningful error messages throughout all phases.

4.2 Achievements of the Project

The following objectives were successfully achieved during the project:

- Designing the HealthScript programming language specification.
- Implementing a lexical analyzer using regular expressions.

- Building a symbol table for identifier management.
- Designing and transforming CFG grammar into LL(1) form.
- Implementing a recursive descent parser.
- Generating parse trees for valid programs.
- Detecting lexical, syntax, and semantic errors.
- Implementing semantic analysis rules.
- Generating intermediate Three-Address Code (TAC).
- Building a complete compiler front-end pipeline.

4.3 Challenges Faced

Several challenges were encountered during the development of the compiler project. One of the main challenges was removing left recursion from the grammar and restructuring expressions to support LL(1) predictive parsing. Another challenge involved handling operator precedence correctly for arithmetic and Boolean expressions.

Additional challenges included implementing semantic checking rules, managing symbol table information, and generating readable TAC instructions while maintaining accurate error reporting with line numbers.

4.4 Future Improvements

Although the HealthScript compiler successfully implements the core front-end phases, several improvements can be added in the future, including:

- Supporting nested scopes and local variables.
- Adding arrays and advanced data structures.
- Implementing optimization techniques for TAC.

- Generating assembly or machine code.
- Supporting function return values and parameter checking.
- Adding graphical visualization for parse trees.
- Improving syntax error recovery mechanisms.

4.5 Final Conclusion

The HealthScript compiler project successfully demonstrates the fundamental concepts of compiler construction, including lexical analysis, syntax analysis, semantic analysis, symbol table management, and intermediate code generation. The project provides a complete and functional compiler front-end capable of analyzing HealthScript programs and generating meaningful outputs and error messages.

Overall, the project achieved its objectives and established a strong foundation for future compiler development and advanced language processing features.

Appendix

The following appendices include the full Python source codes used to implement the HealthScript compiler front-end.

Appendix A: Scanner Source Code

```
import re

def healthscript_scanner(source_code):
    tokens_list = []
    symbol_table = {}
    error_list = []
    total_lexemes = 0

    # Keywords we agreed on in the Language Spec document
    keywords = {
        "start", "finish", "if", "then", "else", "repeat", "while",
"var",
        "int", "float", "string", "bool", "do", "read", "print",
"void",
        "return", "func", "and", "or", "not"
    }

    # Regular expressions for our tokens (Order is critical)
    token_specification = [
        ('BLOCK_COMMENT',      r'\/*(?:.|\\n)*?\/'),          # /*
block comment */
        ('UNTERMINATED_BLOCK', r'\/*(?:.|\\n)*'),              # catch
missing */
        ('LINE_COMMENT',       r'\/\/[^\n]*'),                  # //
single line comment
        ('STRING',              r'"[^\n]*"'),                    # "text
inside quotes"
        ('UNTERMINATED_STRING', r'"[^\n]*'),                      # catch
missing end quote
        ('MALFORMED_FLOAT',     r'[+-]?\\d{9,}\\.|\\d+'),        #
Error: Float exceeding 8 digits
        ('MALFORMED_INTEGER',  r'[+-]?\\d{9,}'),                 #
Error: Integer exceeding 8 digits
        ('FLOAT',               r'[+-]?\\d{1,8}\\.|\\d+'),        #
numbers with decimals (max 8)
        ('INTEGER',             r'[+-]?\\d{1,8}'),                #
normal numbers (max 8)
```

```

        ('REL_OP',                r'==|!=|<=|>=|<|>'),          #
relational operators
        ('ARITH_OP',             r'[\+|-|\*|/%\^=]'),          # math
and assignment (=)
        ('DELIMITER',           r'[\.\(\)\,\{\}\;\:]'),          #
brackets and punctuation
        ('IDENTIFIER',          r'[a-zA-Z][a-zA-Z0-9]*'),        #
variable/function names
        ('NEWLINE',             r'\n'),                        # count
line numbers
        ('WHITESPACE',          r'[\t]+'),                      #
spaces and tabs
        ('UNKNOWN',             r'.'),                          #
anything else is an error
    ]

    tok_regex = '|'.join('(?P<%s>%s)' % pair for pair in
token_specification)
    get_token = re.compile(tok_regex)

    line_num = 1

    for match in get_token.finditer(source_code):
        kind = match.lastgroup
        value = match.group()

        if kind == 'NEWLINE':
            line_num += 1
            continue
        elif kind in ['WHITESPACE', 'LINE_COMMENT']:
            continue
        elif kind == 'BLOCK_COMMENT':
            line_num += value.count('\n')
            continue

        # Handle the deliberate lexical errors
        if kind == 'UNTERMINATED_BLOCK':
            error_list.append(f"Lexical Error at line {line_num}:
Unterminated block comment")
            line_num += value.count('\n')
            continue
        elif kind == 'UNTERMINATED_STRING':
            error_list.append(f"Lexical Error at line {line_num}:
Unterminated string literal [{value}]")
            continue
        elif kind == 'MALFORMED_FLOAT':

```

```

        error_list.append(f"Lexical Error at line {line_num}:
Malformed float number '{value}'")
        continue
    elif kind == 'MALFORMED_INTEGER':
        error_list.append(f"Lexical Error at line {line_num}:
Malformed integer number '{value}'")
        continue
    elif kind == 'UNKNOWN':
        error_list.append(f"Lexical Error at line {line_num}:
Invalid symbol '{value}'")
        continue

    # Handle Identifiers and Keywords
    if kind == 'IDENTIFIER':
        lower_value = value.lower()
        if lower_value in keywords:
            tokens_list.append(("KEYWORD", line_num, lower_value))
            total_lexemes += 1
        elif len(value) > 8:
            error_list.append(f"Lexical Error at line {line_num}:
Identifier '{value}' exceeds max length (8 chars)")
        else:
            tokens_list.append(("IDENTIFIER", line_num, value))
            total_lexemes += 1
            if value not in symbol_table:
                symbol_table[value] = line_num
    else:
        tokens_list.append((kind, line_num, value))
        total_lexemes += 1

    return total_lexemes, tokens_list, symbol_table, error_list

# Testing the Scanner
if __name__ == '__main__':
    # Sample code with perfectly formatted syntax and deliberate
    lexical errors
    test_code = """
    start
    var float ptTemp = 38.5;
    var int heartRt; // Patient HR

    if ptTemp > 37.5 then
        print "Fever";
    finish

    var int veryLongVariableName = 10;

```



```

print "Unclosed string;

var float errorFloat = 123456789.5;
var int errorInt = 987654321;
@
"""

total, tokens, sym_table, errors = healthscript_scanner(test_code)

print("=== Phase 1 Scanner Results ===")
print(f"Total Lexemes Found: {total}")
print()

print("Tokens (<Type, Line, Value>)")
for t in tokens:
    print(f"<{t[0]}, {t[1]}, {t[2]}>")

print()
print("Symbol Table")
for identifier, line in sym_table.items():
    print(f"{identifier} (first seen at line {line})")

print()
print("Errors")
if not errors:
    print("No errors! Ready for Phase 2.")
else:
    for err in errors:
        print(err)

```

Appendix B: Parser Source Code

```
from scanner import healthscript_scanner

class Parser:
    def __init__(self, tokens):
        self.tokens = tokens
        self.current = 0
        self.tree = []

    def current_token(self):
        if self.current < len(self.tokens):
            return self.tokens[self.current]
        return ("EOF", -1, "EOF")

    def add_tree(self, text):
        self.tree.append(text)

    def match(self, expected):
        token_type, line, value = self.current_token()
        if value == expected:
            self.add_tree(f"Matched: {value}")
            self.current += 1
        else:
            raise SyntaxError(
                f"Syntax Error at line {line}: Expected '{expected}'
but found '{value}'"
            )

    def match_type(self, expected_type):
        token_type, line, value = self.current_token()
        if token_type == expected_type:
            self.add_tree(f"Matched: {token_type}({value})")
            self.current += 1
        else:
            raise SyntaxError(
                f"Syntax Error at line {line}: Expected {expected_type}
but found '{value}'"
            )

    def Program(self):
        self.add_tree("<Program>")
        self.match("start")
        self.StmtList()
        self.match("finish")
```

```

def StmtList(self):
    self.add_tree("  <StmtList>")
    while self.current_token()[2] in ["var", "print", "read", "if",
"while", "do", "func"] or self.current_token()[0] == "IDENTIFIER":
        self.Stmt()

def Stmt(self):
    value = self.current_token()[2]
    token_type = self.current_token()[0]

    self.add_tree("    <Stmt>")

    if value == "var":
        self.VarDecl()
    elif token_type == "IDENTIFIER":
        self.AssignStmt()
    elif value == "print":
        self.PrintStmt()
    elif value == "read":
        self.ReadStmt()
    elif value == "if":
        self.IfStmt()
    elif value == "while":
        self.WhileStmt()
    elif value == "do":
        self.FuncCall()
    elif value == "func":
        self.FuncDecl()
    else:
        line = self.current_token()[1]
        raise SyntaxError(f"Syntax Error at line {line}: Unexpected
token '{value}'")

def VarDecl(self):
    self.add_tree("      <VarDecl>")
    self.match("var")
    self.Type()
    self.match_type("IDENTIFIER")

    if self.current_token()[2] == "=":
        self.match("=")
        self.Expr()

    self.match(";")

def Type(self):
    value = self.current_token()[2]

```

```

        line = self.current_token()[1]

        if value in ["int", "float", "string", "bool"]:
            self.add_tree(f"          <Type> {value}")
            self.current += 1
        else:
            raise SyntaxError(f"Syntax Error at line {line}: Expected
data type")

    def ReturnTyp(self):
        value = self.current_token()[2]
        line = self.current_token()[1]

        if value in ["int", "float", "string", "bool", "void"]:
            self.add_tree(f"          <ReturnTyp> {value}")
            self.current += 1
        else:
            raise SyntaxError(f"Syntax Error at line {line}: Expected
return type")

    def AssignStmnt(self):
        self.add_tree("          <AssignStmnt>")
        self.match_type("IDENTIFIER")
        self.match("=")
        self.Expr()
        self.match(";")

    def PrintStmnt(self):
        self.add_tree("          <PrintStmnt>")
        self.match("print")
        self.Expr()
        self.match(";")

    def ReadStmnt(self):
        self.add_tree("          <ReadStmnt>")
        self.match("read")
        self.match_type("IDENTIFIER")
        self.match(";")

    def IfStmnt(self):
        self.add_tree("          <IfStmnt>")
        self.match("if")
        self.BoolExpr()
        self.match("then")
        self.StmntList()

        if self.current_token()[2] == "else":

```

```

        self.match("else")
        self.StmtList()

    self.match("finish")

def WhileStmt(self):
    self.add_tree("        <WhileStmt>")
    self.match("while")
    self.BoolExpr()
    self.match("do")
    self.StmtList()
    self.match("finish")

def FuncDecl(self):
    self.add_tree("        <FuncDecl>")
    self.match("func")
    self.ReturnType()
    self.match_type("IDENTIFIER")
    self.match("(")

    if self.current_token()[2] != ")":
        self.ParamList()

    self.match(")")
    self.StmtList()
    self.match("finish")

def ParamList(self):
    self.add_tree("        <ParamList>")
    self.Type()
    self.match_type("IDENTIFIER")

    while self.current_token()[2] == ",":
        self.match(",")
        self.Type()
        self.match_type("IDENTIFIER")

def FuncCall(self):
    self.add_tree("        <FuncCall>")
    self.match("do")
    self.match_type("IDENTIFIER")
    self.match("(")

    if self.current_token()[2] != ")":
        self.ArgList()

    self.match(")")

```

```

        self.match(";")

    def ArgList(self):
        self.add_tree("        <ArgList>")
        self.Expr()

        while self.current_token()[2] == ",":
            self.match(",")
            self.Expr()

    def BoolExpr(self):
        self.add_tree("        <BoolExpr>")
        self.BoolTerm()

        while self.current_token()[2] == "or":
            self.match("or")
            self.BoolTerm()

    def BoolTerm(self):
        self.add_tree("        <BoolTerm>")
        self.BoolFactor()

        while self.current_token()[2] == "and":
            self.match("and")
            self.BoolFactor()

    def BoolFactor(self):
        self.add_tree("        <BoolFactor>")

        if self.current_token()[2] == "not":
            self.match("not")
            self.BoolFactor()
        else:
            self.Expr()

        if self.current_token()[2] in ["==", "!=", "<", ">", "<=",
">="]:
            self.current += 1
            self.add_tree("Matched: REL_OP")
            self.Expr()

    def Expr(self):
        self.add_tree("        <Expr>")
        self.Term()

        while self.current_token()[2] in ["+", "-"]:
            self.current += 1

```

```

        self.add_tree("Matched: ADD_OP")
        self.Term()

def Term(self):
    self.add_tree("      <Term>")
    self.Power()

    while self.current_token()[2] in ["*", "/", "%"]:
        self.current += 1
        self.add_tree("Matched: MUL_OP")
        self.Power()

def Power(self):
    self.add_tree("      <Power>")
    self.Unary()

    if self.current_token()[2] == "^":
        self.match("^")
        self.Power()

def Unary(self):
    self.add_tree("      <Unary>")

    if self.current_token()[2] in ["+", "-", "not"]:
        op = self.current_token()[2]
        self.match(op)
        self.Unary()
    else:
        self.Factor()

def Factor(self):
    token_type, line, value = self.current_token()
    self.add_tree("      <Factor>")

    if token_type in ["IDENTIFIER", "INTEGER", "FLOAT", "STRING"]:
        self.add_tree(f"Matched: {token_type}({value})")
        self.current += 1
    elif value in ["true", "false"]:
        self.add_tree(f"Matched: BOOL({value})")
        self.current += 1
    elif value == "(":
        self.match("(")
        self.Expr()
        self.match(")")
    else:
        raise SyntaxError(

```

```

        f"Syntax Error at line {line}: Unexpected token
'{value}' in expression"
    )

    def display_parse_tree(self):
        print("\n--- Parse Tree Display ---")
        for item in self.tree:
            print(item)

with open("test.txt", "r") as file:
    source_code = file.read()

total, tokens, sym_table, lexical_errors =
healthscript_scanner(source_code)

if lexical_errors:
    print("Lexical errors found:")
    for err in lexical_errors:
        print(err)
else:
    parser_tokens = [(t[0], t[1], t[2]) for t in tokens]
    parser = Parser(parser_tokens)

    try:
        parser.Program()
        print("Parsing successful.")
        parser.display_parse_tree()
    except SyntaxError as e:
        print(e)

```


Appendix C: Semantic Analyzer and TAC Generator Source Code

```
from scanner import healthscript_scanner

class SemanticCodeGenerator:
    def __init__(self):
        self.symbol_table = {}
        self.semantic_errors = []
        self.tac = []
        self.temp_count = 1

    def new_temp(self):
        temp = f"t{self.temp_count}"
        self.temp_count += 1
        return temp

    def add_symbol(self, name, var_type, line):
        if name in self.symbol_table:
            self.semantic_errors.append(
                f"Semantic Error at line {line}: Duplicate declaration of variable '{name}'"
            )
        else:
            self.symbol_table[name] = {
                "type": var_type,
                "scope": "global",
                "line": line
            }

    def get_type(self, value):
        if value.startswith("'") and value.endswith("'"):
            return "string"
        if value in ["true", "false"]:
            return "bool"
        try:
            int(value)
            return "int"
        except ValueError:
            pass
        try:
            float(value)
            return "float"
        except ValueError:
            pass
        if value in self.symbol_table:
            return self.symbol_table[value]["type"]
```

```

        return "unknown"

    def check_declared(self, name, line):
        if name not in self.symbol_table:
            self.semantic_errors.append(
                f"Semantic Error at line {line}: Variable '{name}' used
but not declared"
            )
            return False
        return True

    def compatible(self, left_type, right_type):
        if left_type == right_type:
            return True
        if left_type == "float" and right_type == "int":
            return True
        return False

    def analyze_expression(self, expr, line):
        parts = expr.replace(";", "").split()

        if len(parts) == 1:
            value = parts[0]
            if value.isidentifier() and value not in ["true", "false"]:
                self.check_declared(value, line)
            return self.get_type(value), value

        if len(parts) == 3:
            left, op, right = parts

            if left.isidentifier() and left not in ["true", "false"]:
                self.check_declared(left, line)
            if right.isidentifier() and right not in ["true", "false"]:
                self.check_declared(right, line)

            left_type = self.get_type(left)
            right_type = self.get_type(right)

            if op == "/" and right == "0":
                self.semantic_errors.append(
                    f"Semantic Error at line {line}: Division by zero"
                )

            if op in ["+", "-", "*", "/", "%", "^"]:
                if left_type not in ["int", "float"] or right_type not
in ["int", "float"]:
                    self.semantic_errors.append(

```

```

        f"Type Error at line {line}: Arithmetic
operation requires numeric values"
    )
    result_type = "float" if "float" in [left_type,
right_type] else "int"

    elif op in ["==", "!", "<", ">", "<=", ">="]:
        result_type = "bool"

    else:
        result_type = "unknown"

    temp = self.new_temp()
    self.tac.append(f"{temp} = {left} {op} {right}")
    return result_type, temp

return "unknown", expr

def process_line(self, line, line_num):
    line = line.strip()

    if not line or line in ["start", "finish"]:
        return

    if line.startswith("var "):
        parts = line.replace(";", "").split()

        if len(parts) >= 3:
            var_type = parts[1]
            name = parts[2]
            self.add_symbol(name, var_type, line_num)

            if "=" in parts:
                eq_index = parts.index("=")
                expr = " ".join(parts[eq_index + 1:])
                expr_type, expr_value =
self.analyze_expression(expr, line_num)

                if not self.compatible(var_type, expr_type):
                    self.semantic_errors.append(
                        f"Type Error at line {line_num}: Cannot
assign '{expr_type}' to variable of type '{var_type}'"
                    )
                self.tac.append(f"{name} = {expr_value}")

    return

```

```

        if line.startswith("print "):
            expr = line.replace("print", "", 1).replace(";",
""").strip()
            expr_type, expr_value = self.analyze_expression(expr,
line_num)
            self.tac.append(f"print {expr_value}")
            return

        if "=" in line and not line.startswith("if"):
            left, right = line.split("=", 1)
            name = left.strip()
            expr = right.replace(";", "").strip()

            if self.check_declared(name, line_num):
                left_type = self.symbol_table[name]["type"]
                expr_type, expr_value = self.analyze_expression(expr,
line_num)

                if not self.compatible(left_type, expr_type):
                    self.semantic_errors.append(
                        f"Type Error at line {line_num}: Cannot assign
'{expr_type}' to variable of type '{left_type}'"
                    )

                self.tac.append(f"{name} = {expr_value}")

            return

        if line.startswith("if "):
            condition = line.replace("if", "", 1).replace("then",
""").strip()
            cond_type, cond_value = self.analyze_expression(condition,
line_num)
            self.tac.append(f"if {cond_value} goto L_true")
            return

        if line.startswith("while "):
            condition = line.replace("while", "", 1).replace("do",
""").strip()
            cond_type, cond_value = self.analyze_expression(condition,
line_num)
            self.tac.append(f"while {cond_value} goto L_loop")
            return

    def analyze(self, source_code):
        lines = source_code.splitlines()

```

```

        for i, line in enumerate(lines, start=1):
            self.process_line(line, i)

        return self.symbol_table, self.semantic_errors, self.tac

# Main execution
with open("test.txt", "r") as file:
    source_code = file.read()

total, tokens, sym_table, lexical_errors =
healthscript_scanner(source_code)

print("=== Phase 3: Semantic Analysis and Code Generation ===")
print()

if lexical_errors:
    print("Lexical errors found:")
    for err in lexical_errors:
        print(err)
else:
    analyzer = SemanticCodeGenerator()
    symbol_table, semantic_errors, tac = analyzer.analyze(source_code)

    print("--- Enhanced Symbol Table ---")
    for name, info in symbol_table.items():
        print(
            f"{name} | Type: {info['type']} | Scope: {info['scope']} |
Declared at line: {info['line']}"
        )

    print()
    print("--- Semantic Errors ---")
    if semantic_errors:
        for err in semantic_errors:
            print(err)
    else:
        print("No semantic errors found.")

    print()
    print("--- Generated Three-Address Code (TAC) ---")
    for instruction in tac:
        print(instruction)

    print()
    if not semantic_errors:

```

```
        print("End-to-end compilation completed successfully.")  
    else:  
        print("Compilation stopped due to semantic errors.")
```